



**COMSATS University Islamabad,  
COMSATS Road, off GT Road, Sahiwal, Pakistan**

## **Management Suite (NEXORA)**

**A project presented to  
COMSATS Institute of Information Technology, Islamabad**

**In partial fulfillment  
of the requirement for the degree of**

***Bachelor of Science in Computer Science (2022-2026)***

**By**

**Muhammad Abdullah CIIT/SP22-BCS-060/SWL**

# DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

**Muhammad Abdullah**

---

# CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “**NEXORA**” was developed by **Muhammad Abdullah (CIIT/SP22-BCS/C/060)** under the supervision of **Ms. Azka Riaz** and co supervisor **Mr Ali Sher Kashif** and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

-----  
**Supervisor**

-----  
**External Examiner**

-----  
**Head of Department**  
**(Department of Computer Science)**

# Executive Summary

In public places, there is often a need for monitoring people and different activities going on, which can be referred later for many reasons including security. Appointing humans for this task involves many problems such as increased employee hiring, accuracy problem, trust, no proof for later use, and also the fact that a human can remember things till a certain time limit. Talking about the current security system, they use dumb still cameras with a continuous recording facility irrespective of the fact that any event may happen or not. Moreover they are usually pointing at a specific user defined locations so more than one cameras are required to cover the entire region.

To prevent all these problems from prevailing, the CSCS is developed. It is a surveillance system, which provides solution to many of these problems. It is a stand-alone application which doesn't require any computer to operate. It monitors different situations using a camera which is able to rotate intelligently based on sensor messages and captures the scene in the form of video or photos later reference as well.

**Customizable Surveillance Control System (CSCS)** is a surveillance system that can be assigned a sensor type as in our case a heat sensor is used, it works accordingly, rotates the camera upon event detection and perform user defined actions like capturing video and stores them, for the future use.

It is an embedded system consisting of Linux fox kit with embedded a running server application also a camera, USB storage device and a sensor node base station is attached with fox kit. LAN communication is used by user to download the videos and to operate the system manually.

# Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Ms. Azka Riaz” and our Co-Supervisor “Mr. Ali Sher Kashif”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

**Muhammad Abdullah**

---

# Abbreviations

|             |                                   |
|-------------|-----------------------------------|
| <b>SRS</b>  | Software Require Specification    |
| <b>PC</b>   | Personal Computer                 |
| <b>UI</b>   | User Interface                    |
| <b>UX</b>   | User Experience                   |
| <b>API</b>  | Application Programming Interface |
| <b>CRUD</b> | Create, Read, Update, Delete      |
| <b>JS</b>   | Java Script                       |
| <b>JSON</b> | JavaScript Object Notation        |
| <b>CSS</b>  | Cascading Style Sheets            |
| <b>HTML</b> | HyperText Markup Language         |

# Table of Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Chapter 1 .....</b>                                      | <b>1</b>  |
| <b>Introduction.....</b>                                    | <b>1</b>  |
| 1 Introduction.....                                         | 2         |
| 1.1 Brief .....                                             | 2         |
| 1.2 Relevance to Course Modules .....                       | 3         |
| 1.3 Project Background .....                                | 4         |
| 1.4 Literature Review .....                                 | 6         |
| 1.5 Analysis from Literature Review.....                    | 8         |
| 1.6 Methodology & Software Lifecycle for This Project ..... | 9         |
| <b>Chapter 2 .....</b>                                      | <b>11</b> |
| <b>Problem Definition.....</b>                              | <b>11</b> |
| 2 Problem Definition.....                                   | 12        |
| 2.1 Problem Statement.....                                  | 12        |
| 2.2 Deliverables and Development Requirements.....          | 13        |
| 2.2.1 Deliverables .....                                    | 13        |
| 2.2.2 Development Requirements.....                         | 14        |
| 2.3 Current System .....                                    | 15        |
| <b>Chapter 3 .....</b>                                      | <b>17</b> |
| <b>Requirement Analysis.....</b>                            | <b>17</b> |
| 3 Requirement Analysis.....                                 | 18        |
| 3.1 Use Case Diagram(s) .....                               | 18        |
| 3.2 Detailed Use Case.....                                  | 18        |
| 3.3 Functional Requirements .....                           | 26        |
| 3.4 Non-Functional Requirements.....                        | 27        |
| <b>Chapter 4 .....</b>                                      | <b>30</b> |
| <b>Design And Architecture .....</b>                        | <b>30</b> |
| 4 Design and Architecture .....                             | 31        |
| 4.1 System Architecture.....                                | 31        |
| 4.2 Data Dictionary.....                                    | 34        |
| 4.3 Process Flow / Representation.....                      | 34        |
| 4.4 Design Models .....                                     | 35        |
| 4.4.1 Class Diagram.....                                    | 35        |
| 4.4.2 ERD Diagram .....                                     | 36        |
| 4.4.3 Sequence Diagram .....                                | 37        |
| 4.4.4 State Transition Diagram .....                        | 38        |

|                                         |           |
|-----------------------------------------|-----------|
| 4.4.5 Data Flow Diagram.....            | 39        |
| <b>Chapter 5 .....</b>                  | <b>43</b> |
| <b>Implementation .....</b>             | <b>43</b> |
| 5 Implementation .....                  | 44        |
| 5.1 Algorithms .....                    | 45        |
| 5.2 External APIs.....                  | 47        |
| 5.3 User Interface.....                 | 48        |
| <b>Chapter 6 .....</b>                  | <b>57</b> |
| <b>Testing and Evaluation.....</b>      | <b>57</b> |
| 6 Testing and Evaluation .....          | 58        |
| 6.1 Manual Testing .....                | 58        |
| 6.1.1 System Testing.....               | 58        |
| 6.1.2 Unit Testing.....                 | 59        |
| 6.1.3 Functional Testing.....           | 62        |
| 6.1.4 Integration Testing .....         | 65        |
| 6.2 Automated Testing.....              | 67        |
| <b>Chapter 7 .....</b>                  | <b>69</b> |
| <b>Conclusion and Future Work .....</b> | <b>69</b> |
| 7 Conclusion And Future Work.....       | 70        |
| 7.1 Conclusion .....                    | 70        |
| 7.2 Future Work.....                    | 71        |
| <b>References.....</b>                  | <b>73</b> |



# List of Figures

|                                                          |    |
|----------------------------------------------------------|----|
| Figure 1 Use Case Diagram for Nexora .....               | 18 |
| Figure 2 System Architecture .....                       | 32 |
| Figure 3 MVVM Architecture .....                         | 32 |
| Figure 4 Activity Diagram .....                          | 35 |
| Figure 5 Class Diagram .....                             | 36 |
| Figure 6 ERD Diagram .....                               | 37 |
| Figure 7 Sequence Diagram.....                           | 38 |
| Figure 8 State Transition Diagram.....                   | 39 |
| Figure 9 DFD level 0 .....                               | 40 |
| Figure 10 DFD Level 1 .....                              | 41 |
| Figure 11 DFD Level 2 .....                              | 42 |
| Figure 12 Splash Screen .....                            | 48 |
| Figure 13 Login Screen .....                             | 49 |
| Figure 14 Sign Up Screen.....                            | 49 |
| Figure 15 Dashboard.....                                 | 50 |
| Figure 16 Invoice Creation .....                         | 50 |
| Figure 17 Sales and Record .....                         | 51 |
| Figure 18 Ledger History.....                            | 51 |
| Figure 19 Admin Panel EMS .....                          | 52 |
| Figure 20 Reports .....                                  | 53 |
| Figure 21 Leave Requests.....                            | 53 |
| Figure 22 Intelligent Employee Performance Scoring ..... | 54 |
| Figure 23 User Payslip.....                              | 54 |
| Figure 24 Achievements .....                             | 55 |
| Figure 25 User Attendance .....                          | 55 |
| Figure 26 Announcements .....                            | 56 |

# List of Tables

|                                                                |    |
|----------------------------------------------------------------|----|
| Table 1:Analysis .....                                         | 8  |
| Table 2 Login .....                                            | 19 |
| Table 3:Get Registered.....                                    | 19 |
| Table 4:View System Modules .....                              | 20 |
| Table 5: Company/Client Details.....                           | 20 |
| Table 6: View Item/Service Prices .....                        | 21 |
| Table 7:Create Invoice .....                                   | 22 |
| Table 8: Mark Attendance (EMS Module) .....                    | 22 |
| Table 9: Add Employee.....                                     | 23 |
| Table 10:Assign Task.....                                      | 24 |
| Table 11:Manage organizational roles .....                     | 24 |
| Table 12: View Dashboard.....                                  | 25 |
| Table 13 Download Invoice Pdf .....                            | 26 |
| Table 14: Data Dictionary.....                                 | 34 |
| Table 15: External APIs .....                                  | 47 |
| Table 16 Unit Testing 1: Login .....                           | 59 |
| Table 17 Unit Testing 2: Add Employee.....                     | 59 |
| Table 18 Unit Testing 3: Attendance Marking.....               | 60 |
| Table 19 Unit Testing 4: Create Invoice .....                  | 60 |
| Table 20 Unit Testing 5: Task Assignment. ....                 | 60 |
| Table 21 Unit Testing 6: Payroll Calculation.....              | 61 |
| Table 22 Unit Testing 7: Client Management .....               | 61 |
| Table 23 Unit Testing 8: Product/ Service Management .....     | 61 |
| Table 24 Unit Testing 9: Task Status Update .....              | 62 |
| Table 25 Functional Testing 1: Login Module.....               | 62 |
| Table 26 Functional Testing 2: Invoice Generation Module ..... | 62 |
| Table 27 Functional Testing 3: Attendance Module .....         | 63 |
| Table 28 Functional Testing 4: Task Assignment Module .....    | 63 |
| Table 29 Functional Testing 5: Client Management Module.....   | 63 |
| Table 30 Functional Testing 6: Product/Service Module.....     | 64 |
| Table 31 Functional Testing 7: Payroll Module.....             | 64 |
| Table 32 Functional Testing 8: Dashboard Module .....          | 64 |
| Table 33 Functional Testing 9: File Upload Module .....        | 65 |
| Table 34 Automated Testing .....                               | 67 |

# **Chapter 1**

## **Introduction**

# 1 Introduction

Digital transformation has become a fundamental necessity for modern businesses, especially small and medium enterprises (SMEs) that rely heavily on efficient operations, accurate financial management, and streamlined employee workflows. As organizations expand and their departments grow, managing tasks such as invoicing, employee monitoring, attendance tracking, payroll calculation, and internal coordination becomes increasingly challenging. Traditional paper-based or manually maintained systems often lead to errors, delays, data loss, miscommunication, and reduced organizational productivity. To overcome these limitations and ensure seamless digital operations, Nexora Business Suite has been developed as an integrated, cloud-enabled software solution built to modernize business management processes.

Nexora is a dual-module business suite containing:

Nexora Invoice System, a MERN-based invoicing platform designed for creating, managing, tracking, and storing digital invoices with accuracy and security; and Nexora Employee Management System, a MongoDB-based cloud solution for managing attendance, tasks, payroll, employee records, and internal workflow automation. Together, these modules form a unified digital ecosystem that enables businesses to reduce paperwork, optimize time, enhance transparency, and maintain organized digital records.

Unlike traditional standalone systems, Nexora provides an integrated experience where both financial operations and HR workflows are digitized under one platform. The Invoice module allows businesses to generate detailed invoices, maintain client records, monitor due payments, and track financial activities through a modern web interface. Meanwhile, the Employee Management System automates daily attendance, allows task assignment, manages payroll based on attendance data, and maintains real-time employee information using a centralized MongoDB database architecture. By connecting these modules, Nexora ensures that day-to-day business operations run smoothly without relying on separate tools or manual spreadsheets.

Nexora follows a modular, iterative development approach that allows incremental enhancement of features, rapid integration of improvements, and continuous refinement based on real-world use cases. Built with a strong focus on usability, accuracy, automation, and security, Nexora aims to provide SMEs with a cost-effective, scalable, and user-friendly digital ecosystem. The project ultimately replaces traditional administrative processes with a centralized platform capable of modernizing business workflows, improving data integrity, minimizing manual effort, and ensuring efficient organizational management.

## 1.1 Brief

Nexora is a modern and integrated business management suite designed to streamline, automate, and centralize essential organizational operations within a single digital ecosystem. Built with a focus on efficiency, scalability, and ease of use, the system provides businesses with a secure, cloud-based platform for handling both financial activities and internal workforce processes. By combining two powerful modules under one unified system, Nexora significantly reduces manual workload and enhances operational transparency.

The suite merges a MERN-based Invoice Management System with a MongoDB-powered Employee Management System, allowing organizations to manage a wide range of activities from a single interface. The invoice module supports client management, product cataloging,

automated calculations, sales reporting, ledger monitoring, and real-time financial tracking. On the other hand, the employee management module enables seamless handling of attendance, task assignments, leave requests, performance monitoring, and payroll generation.

Through this integrated approach, Nexora ensures that businesses—whether small, medium, or growing—can efficiently oversee invoices, customers, employees, and day-to-day administrative workflows without switching between multiple applications. The platform's smart automation and centralized data synchronization not only improve accuracy and record-keeping but also support informed decision-making, making Nexora a comprehensive solution for modern business operations.

The suite integrates multiple modules including:

- Digital invoice creation and management
- Client and payment tracking
- Automated attendance monitoring
- Task assignment and employee workflow management
- Payroll calculation based on attendance records
- Real-time data synchronization using MongoDB
- Secure role-based access for admin and employees
- Dashboard analytics for business insights

The entire system is developed using React JS, Node.js, Express, MongoDB, JWT-based Authentication, REST APIs, and centralized database services. The development process follows the Iterative Software Process Model, allowing incremental enhancement of modules, continuous testing, and refinement of business workflows. Nexora aims to replace traditional manual record-keeping with a centralized, efficient, and scalable digital platform tailored for small and medium-sized enterprises.

## **1.2 Relevance to Course Modules**

The development of the Nexora Business Suite utilizes knowledge from several BSCS courses:

### **Course-Wise Relevance**

#### **Software Engineering – I & II**

- Requirement gathering, SRS, SDD documentation
- Use cases, UML diagrams, DFDs, class diagrams
- Software Development Life Cycle (SDLC) models
- Design principles and modular system design
- Architecture modeling for MERN-based multi-module systems
- Nexora strictly follows SE documentation standards

#### **Web Development**

- React-based UI development
- Component-driven architecture
- REST APIs integration with Express.js

- Form handling, validation, and routing
- Responsive design for dashboard and modules

### **Database Systems**

- MongoDB NoSQL modeling (Invoice, EMS, Pet Store modules)
- Centralized MongoDB Atlas database design
- Data modeling for invoices, employees, attendance, tasks, payroll
- Database indexing, schema design, and relationships
- CRUD operations and query optimization

### **Object-Oriented Programming**

- Class structures (User, Invoice, Employee, Task, Payroll)
- Method-based interactions (createInvoice, markAttendance, assignTask, updatePayroll)
- Encapsulation of business logic in controllers and services
- Reusable components & modular functions

### **Information Security**

- User authentication using JWT-based security
- Secure access control for admin and employees
- Role-based permissions across all modules
- Data validation & protection of sensitive records
- Secure API consumption

### **Human-Computer Interaction**

- User-centered interface design
- Minimal learning curve for employees and admin users
- Dashboard-based navigation for better usability
- Clean, intuitive layout for web app interactions

### **Computer Networks**

- Client–server communication via REST APIs
- Asynchronous API handling and data exchange
- Network state handling, async operations, API calls
- Distributed access through centralized database

### **Distributed / Cloud Computing**

- MongoDB Atlas cloud database deployment
- Centralized data storage for all modules
- Cloud-based invoice and HR record management
- Remote accessibility across devices

### **Software Project Management**

- Iterative development model
- Task breakdown, module-wise development
- Time estimation, risk evaluation, version control (GitHub)

## **1.3 Project Background**

In recent years, businesses of all sizes especially small and medium enterprises (SMEs) have increasingly transitioned toward digital operations to improve efficiency, reduce costs, and

maintain accurate record-keeping. As organizations expand, managing daily business activities such as invoicing, employee attendance, payroll processing, task assignments, and internal coordination becomes progressively more complex. Traditional methods such as handwritten invoices, physical registers, spreadsheets, and manually calculated payroll systems are not only time-consuming but also highly prone to errors, data loss, miscommunication, and inefficiency. Most businesses continue to rely on separate tools for HR management and financial operations. For example, many use one application for attendance, another for payroll, and yet another for invoicing. This fragmented approach results in:

- Lack of integration between departments
- Repetitive manual data entry
- Increased chances of human error
- Difficulty in tracking employee performance
- Poor financial visibility
- Inefficiencies in day-to-day operations

Moreover, traditional invoice systems often fail to provide automated tracking, client management, overdue reminders, or centralized financial history. On the HR side, manual attendance registers, unstructured employee records, and outdated payroll methods lead to inaccurate salary calculations and reduced organizational productivity.

To address these challenges, Nexora Business Suite has been conceptualized as a unified, modern, and fully digital solution. Nexora integrates two essential modules—Invoice Management System (MERN-based) and Employee Management System (MongoDB-based)—within a single business platform. This hybrid approach combines the power of MongoDB, Express, React and Node.js, to deliver a seamless, scalable, and cloud-enabled ecosystem for businesses.

The idea behind Nexora is to eliminate the dependency on scattered tools and outdated manual practices by introducing an all-in-one suite where businesses can manage

- Invoices
- Employees
- Tasks
- Attendance
- and payroll from one place.

Nexora aims to provide

- Automation
- Accuracy
- Transparency
- and real-time updates features that are essential for modern digital organizations.

By digitizing operational workflows and centralizing business data, Nexora helps organizations reduce administrative workload, improve decision-making, increase productivity, and maintain reliable long-term records. The project aligns with the growing global trend of business automation and supports modern enterprises in their shift toward efficient, sustainable, and digitally mature environments.

## **1.4 Literature Review**

A review of existing literature highlights several significant developments, emerging trends, and ongoing challenges in modern business management systems, particularly in the domains of invoicing platforms, financial automation tools, employee tracking technologies, and cloud-based workflow environments. Researchers consistently emphasize the transformation from traditional, manual record-keeping methods toward intelligent, real-time digital systems that enhance accuracy, reduce administrative workload, and provide greater financial visibility for organizations.

Within invoicing and financial automation, studies underline the growing use of cloud databases, API-driven architectures, automated tax and subtotal calculations, and intelligent billing workflows that enable businesses to manage transactions with minimal human intervention. Literature also notes the adoption of smart dashboards and analytics tools that support strategic decision-making through predictive insights and trend analysis.

In the context of employee management, research highlights the shift toward centralized attendance automation, biometric or digital check-ins, integrated payroll computation, and real-time task monitoring tools that improve workforce productivity and operational consistency. Modern systems focus not only on capturing data but also on interpreting it to support better HR decisions.

Despite these advancements, the literature identifies persistent challenges such as maintaining data security, ensuring scalability for growing organizations, achieving multi-user synchronization in cloud environments, and designing intuitive interfaces accessible to both administrative and non-technical staff. These insights collectively demonstrate the increasing need for integrated, intelligent, and user-centric management solutions like Nexora that address both financial and human-resource-related workflows within a unified digital ecosystem.

### **1. Digital Invoicing Systems**

Recent studies emphasize a major transformation in financial workflows as businesses shift from traditional, manual invoicing toward fully automated and cloud-based invoicing systems. Digital invoicing significantly minimizes human error by automating calculations, tax application, and total cost estimation, ensuring greater accuracy and consistency across financial documents. Literature highlights that cloud-enabled invoice systems allow secure storage of records with timestamps, enabling organizations to maintain transparent financial histories and meet auditing requirements more easily. Researchers also stress the value of advanced features such as real-time payment tracking, automated reminders, standardized invoice templates, client purchase history, and integration with analytics tools. These capabilities not only streamline financial operations but also improve decision-making, accelerate billing cycles, and enhance overall business efficiency.

### **2. Attendance & Workforce Management**

Workforce automation has become a major area of focus in modern HR and organizational research, as businesses increasingly move toward digital solutions to improve efficiency and accuracy. Recent studies highlight that cloud-based attendance tracking systems allow real-time monitoring of employee presence, reducing manual errors that commonly occur with paper-based or manual systems. Many advanced solutions now incorporate biometric verification,



facial recognition, and GPS-based mobile check-ins to prevent proxy attendance and ensure authenticity.

Research further indicates that automated attendance systems significantly improve punctuality monitoring by maintaining accurate logs of check-in and check-out times without manual intervention. These systems also support real-time performance tracking by correlating attendance data with task completion, productivity records, and overall employee behavior. Several HR-focused studies emphasize that when attendance data is directly integrated with payroll modules, salary calculations become faster, more accurate, and free from manual discrepancies. This integration reduces administrative workload, minimizes payroll disputes, and enhances organizational transparency.

### **3. Payroll Automation**

Payroll management research highlights the growing importance of integrating attendance records, working hours, overtime logs, salary rules, and deductions into a unified automated system to ensure accuracy and fairness. Studies show that traditional spreadsheet-based payroll processes are not only time-consuming but also vulnerable to calculation mistakes, missing entries, and human bias. Modern research strongly favors cloud-connected payroll engines that automatically pull verified data from attendance modules, reducing manual intervention and ensuring consistent salary computation every month. Automated systems also improve transparency by keeping complete audit trails, making it easier for organizations to review revisions, verify calculations, and comply with financial regulations. Furthermore, literature emphasizes the role of automation in preventing disputes, reducing administrative workload, supporting real-time payroll updates, and enhancing overall organizational efficiency.

### **4. Task & Employee Workflow Automation**

Recent advancements in business process automation emphasize the growing importance of digital task assignment and workflow management tools in modern organizations. Researchers argue that centralized task management significantly enhances communication by allowing all stakeholders to track responsibilities, deadlines, and progress in one unified system. Studies further show that real-time task updates reduce delays and misunderstandings, ensuring that employees receive immediate notifications about new assignments, status changes, or pending actions. Literature also highlights the value of role-based access control, which ensures that only authorized users can modify or approve tasks, thereby improving security and accountability. Additionally, activity logs and audit trails are identified as essential components for monitoring performance, identifying bottlenecks, and maintaining transparency within teams. These features collectively contribute to improved organizational coordination, better resource allocation, and higher overall productivity.

### **5. Cloud-Based Business Applications**

Modern literature consistently highlights the widespread adoption of cloud platforms such as Firebase, Firestore, and MongoDB due to their scalability, security, and global accessibility. Researchers emphasize that these platforms offer built-in real-time synchronization, allowing multiple users across different locations to access and update data simultaneously without conflicts. Cloud databases also reduce the need for physical servers and maintenance teams,

significantly lowering infrastructure and operational costs for organizations. Studies point out that the high availability and automatic backup mechanisms in cloud systems enhance reliability and prevent data loss during failures. Additionally, the flexibility of schema-less NoSQL models enable faster development cycles and easier handling of dynamic business data. Literature further stresses that cloud platforms support seamless integration with APIs, authentication services, and distributed applications, making them ideal for modern business solutions that require continuous connectivity and cross-platform collaboration.

**6. Integrated Business Suites**

Recent research indicates a strong movement toward unifying multiple organizational functions into a single, cohesive management system. Instead of operating separate applications for invoicing, HR, payroll, attendance, and workflow coordination, modern businesses increasingly adopt integrated business suites that centralize all operations under one platform. Literature shows that such unified systems significantly enhance efficiency by eliminating duplicated data, reducing manual synchronization, and minimizing errors caused by switching between multiple tools. Researchers also highlight that integrated platforms improve data consistency, allowing different departments to access the same real-time information, which strengthens collaboration and communication across the organization. Moreover, centralized systems enable more powerful analytics, as consolidated data makes it possible to generate comprehensive performance insights and actionable reports. Studies further emphasize that unified solutions reduce overall maintenance costs, simplify IT infrastructure, and provide a more seamless user experience for both administrators and employees.

**1.5 Analysis from Literature Review**

The literature review clearly demonstrates that modern organizations are rapidly transitioning toward automated, cloud-driven management systems that unify financial operations and workforce administration. Studies consistently highlight the limitations of traditional manual invoicing, payroll processing, and employee tracking—particularly in terms of accuracy, transparency, and scalability. Research also shows that cloud technologies such as MongoDB provide real-time synchronization, improved security, and global accessibility, making them ideal for integrated business platforms like Nexora. Furthermore, existing literature confirms a growing preference for systems that offer automated calculations, smart analytics, centralized dashboards, and role-based access control. These findings validate the need for a unified business suite that combines invoicing, attendance tracking, employee management, payroll automation, and task monitoring into one cohesive environment. Nexora directly aligns with these identified gaps by offering a modern, cloud-powered, all-in-one solution designed to overcome the shortcomings of fragmented tools.

*Table 1:Analysis*

| Existing Solutions | Limitation |
|--------------------|------------|
|                    |            |

|                                                  |                                                                                                                   |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| QuickBooks (Invoicing + Accounting)              | Expensive subscription, complex interface for SMEs, lacks integrated employee attendance module                   |
| Zoho Books & Zoho People                         | Separate modules for HR and invoicing; integration is not seamless; requires add-ons; costly for small businesses |
| Wave Invoicing                                   | No employee management, no payroll automation, limited reporting features                                         |
| HRM Cloud Platforms (e.g., BambooHR)             | Provide HR features only; no invoice management; over-featured and costly for small teams                         |
| Standalone Attendance Apps                       | Only mark attendance; no payroll integration; no task assignment or employee profiling                            |
| Manual Record-Keeping (Registers & Spreadsheets) | High chances of errors, lacks automation, no centralized data, no real-time insights                              |

**Nexora Management Suite** overcomes these limitations by offering a unified, cloud-based, customizable, and real-time invoicing module combined with an integrated Employee Management module.

## 1.6 Methodology & Software Lifecycle for This Project

The system consists of two major independent modules (Invoice System & Employee Management System), which can be developed and improved separately.

Each module can be progressively enhanced with new features such as advanced reporting, payroll automation, and real-time business insights.

The iterative model allows continuous integration of user feedback after each cycle, improving usability and performance.

It provides flexibility for integrating cloud services (MongoDB), analytics, and future automation features without redesigning the entire system.

The modular nature of Nexora benefits from iterative cycles where design, development, testing, and deployment occur repeatedly until the system is refined.

This approach ensures that both the Invoice and RMS modules remain scalable, maintainable, and adaptable to evolving business requirements.

### 1.6.1 Rationale Behind Selected Methodology

The iterative model was chosen because:

- It reduces development risk by allowing early detection of issues in both the Invoice and Employee Management modules.

- It enables continuous refinement of features such as attendance tracking, payroll automation, and invoice generation.
- It supports ongoing testing after each cycle, ensuring higher reliability and stability before moving to the next phase.
- It allows the creation of early working prototypes for both modules, helping gather feedback from potential users and improving usability.
- It fits the hybrid architecture of MERN (for Invoice Module) and Firebase (for RMS Module), where updates and improvements need frequent integration.
- It supports seamless expansion of the system by allowing the addition of new modules such as reporting dashboards, analytics, and business insights in future iterations.

# **Chapter 2**

## **Problem Definition**

## 2 Problem Definition

The chapter also examines the limitations of existing digital solutions, which typically offer isolated functionalities—for example, separate tools for invoicing, attendance, payroll, or task management. Such fragmentation forces organizations to switch between multiple platforms, resulting in data inconsistency, duplication of work, and poor coordination across departments. Additionally, many current systems lack real-time synchronization, automated analytics, and cloud-based accessibility, making it difficult for businesses to maintain transparency and make data-driven decisions. These gaps highlight the need for a unified, scalable, and intelligent business platform. Nexora is designed to bridge this gap by integrating invoicing, employee management, attendance tracking, payroll automation, task monitoring, and reporting within a single cloud-powered environment. This chapter further outlines the motivation behind developing Nexora as an all-in-one ecosystem for SMEs, along with the core deliverables and technical requirements necessary for building a reliable, modern, and fully synchronized business management solution.

### 2.1 Problem Statement

In today's business landscape, SMEs are required to manage multiple operations such as invoicing, employee attendance, payroll, and task coordination. However, most organizations still rely on outdated manual methods or disconnected digital tools, which leads to inefficiency, errors, delays, and poor workflow visibility.

Traditional systems face several limitations, including:

Manual invoice creation causing miscalculations, errors, and duplicated financial records.

Paper-based attendance prone to manipulation, data loss, and inaccurate reporting.

Time-consuming payroll processing, especially when derived manually from attendance logs.

Scattered employee data stored in separate files, making record-keeping inconsistent.

Lack of integrated task management, resulting in miscommunication between admin and staff.

Missing automation features such as real-time updates, centralized analytics, and auto-generated reports.

Difficulty tracking overdue invoices, payments, or pending tasks due to lack of digital monitoring.

No unified system, forcing businesses to rely on multiple unlinked HR and invoicing tools.

Existing solutions often remain costly, isolated, or incomplete, lacking support for:

- Combined invoicing + HR workflows
- Payroll automation linked with attendance
- Centralized cloud dashboards
- Real-time updates
- Role-based secure access for admin and employees

These challenges create the need for a single, affordable, integrated, cloud-powered business suite that connects HR and financial operations under one platform.

The **Nexora Business Suite** addresses these challenges by offering:

- Cloud-based invoice management and client tracking
- Real-time MongoDB attendance monitoring

- Automated payroll directly connected to attendance
- Task assignment and workflow automation
- Employee record management in a centralized system
- Secure authentication & role-based permissions
- Unified dashboards for HR + financial insights

Nexora enables businesses to shift from scattered, manual processes to a modern, accurate, and automated digital ecosystem.

## **2.2 Deliverables and Development Requirements**

This section outlines the major deliverables and technical prerequisites required for the successful development and deployment of the Nexora Management Suite. It identifies the core software components, system modules, and documentation that will be produced as part of the project. Additionally, it highlights the technical, hardware, and software requirements necessary to ensure smooth implementation, scalability, and long-term maintainability of the system. These deliverables form the foundation for building a secure, cloud-based, and fully integrated business management solution.

### **2.2.1 Deliverables**

#### **1. Web-Based Business Suite (Nexora)**

##### **A. Invoice Management System (MERN Stack)**

- Digital invoice creation
- Employee profile management
- Real-time MongoDB-based attendance
- Check-in / Check-out logging
- Automated payroll calculation
- Task assignment and status tracking
- Leave request handling
- Dashboard analytics (attendance, tasks, payroll)
- Admin/Employee role separation

##### **B. Employee Management System**

- Employee profile management
- Real-time MongoDB-based attendance
- Check-in / Check-out logging
- Automated payroll calculation
- Task assignment and status tracking
- Leave request handling
- Dashboard analytics (attendance, tasks, payroll)
- Admin/Employee role separation

#### **2. Cloud Backend Services**

##### **A. MERN Backend (Invoice Module)**

- Node.js + Express REST APIs

- MongoDB Atlas database
- JWT-based authentication
- Invoice, client, and financial records management

#### **B. MongoDB Backend (RMS Module)**

- MongoDB Authentication (Email/Password)
- MongoDB Cloud Database
- Real-time updates for attendance & tasks
- MongoDB Storage for employee files (optional)
- Security rules & access control

### **3. Database Structure & Rules**

- MongoDB schema design for invoices & clients
- MongoDB collections for employees, tasks, attendance, and payroll
- Role-based access rules
- Cloud security rules for safe data operations

### **4. Documentation Deliverables**

- Software Requirements Specification (SRS)
- Software Design Document (SDD)
- Final Year Project Report
- User Manual / System Usage Guide

## **2.2.2 Development Requirements**

### **1. Technical Requirements**

#### **Frontend:**

- React JS (Invoice + Admin Dashboard UI)
- React Context API / Hook
- Tailwind / CSS (as implemented)

#### **Backend:**

- Node.js, Express.js (Invoice Module)
- MongoDB + Authentication (Employee Module)

#### **Databases**

- MongoDB Atlas (Invoice System)
- MongoDB Atlas (RMS)

#### **APIs & Integrations**

- Axios for API communication
- MongoDB SDKs
- JWT for secure login
- PDF generation (for invoices)

#### **Development Tools:**

- Visual Studio Code



- MongoDB Console
- MongoDB Atlas
- Postman (API testing)
- Git/GitHub for version control

## **2. Hardware Requirements**

- Laptop/PC with minimum 8GB RAM
- Stable internet connection
- Any modern browser (Chrome recommended)

## **3. Software Requirements**

- Node.js (LTS version)
- React environment
- MongoDB Atlas account
- VS Code extensions for React/ESLint

## **2.3 Current System**

Before the development of Nexora Business Suite, most businesses—especially small and medium-sized enterprises—relied on manual or partially digitized processes for handling invoices, attendance, employee records, payroll calculations, and task assignments. In traditional setups, invoices were often created in Excel sheets or printed on paper, manually calculated, and physically shared with clients. This approach frequently led to calculation errors, missing records, delayed payments, and difficulties in tracking outstanding invoices.

Employee management also depended heavily on paper-based attendance registers, manual timesheets, or basic mobile apps with no integration to payroll. Attendance manipulation, missing logs, and human errors were common issues. Payroll was calculated manually at the end of every month using spreadsheets, which required cross-verifying attendance, deductions, and leaves—making the process slow and prone to mistakes. Similarly, task assignments were communicated informally through verbal instructions or messaging apps, causing miscommunication, lost instructions, and no proper tracking mechanism. Employee files and records were often stored in physical folders or scattered across different systems, making it difficult to maintain consistency and accuracy.

Although certain standalone digital tools exist—like online invoicing applications, attendance trackers, or HR management platforms—they are not designed to work as a unified ecosystem. Businesses often require multiple separate systems to handle invoices, employee data, attendance, or payroll. These disconnected tools lack seamless integration, resulting in repeated data entry, inconsistent records, and time-consuming switching between multiple platforms. Many existing solutions are also costly, overly complex for small organizations, or missing essential features such as automated payroll linkage, cloud-based attendance logs, or centralized business analytics.

Nexora Business Suite addresses these limitations by offering an integrated, cloud-supported platform that combines invoicing and HR workflows into a single system. It provides automated invoice generation, real-time attendance logging using Firestore, payroll calculation linked directly to attendance data, centralized employee record management, and structured task

assignments. By replacing manual processes with a unified business suite, Nexora ensures faster operations, accurate data handling, improved coordination, and better decision-making for modern organizations.

# **Chapter 3**

## **Requirement Analysis**

### 3 Requirement Analysis

This chapter summarizes the key functional and non-functional requirements of the Nexora Management Suite based on the SRS. It defines core system features for the Invoice Management System and Employee Management System, including invoicing, attendance, payroll, task management, and reporting, all supported through a centralized MongoDB database to ensure scalability, reliability, and data consistency.

#### 3.1 Use Case Diagram(s)

The Use Case Diagram presents a high-level view of interactions between Admin, Employee, and Client/User within the Nexora system.

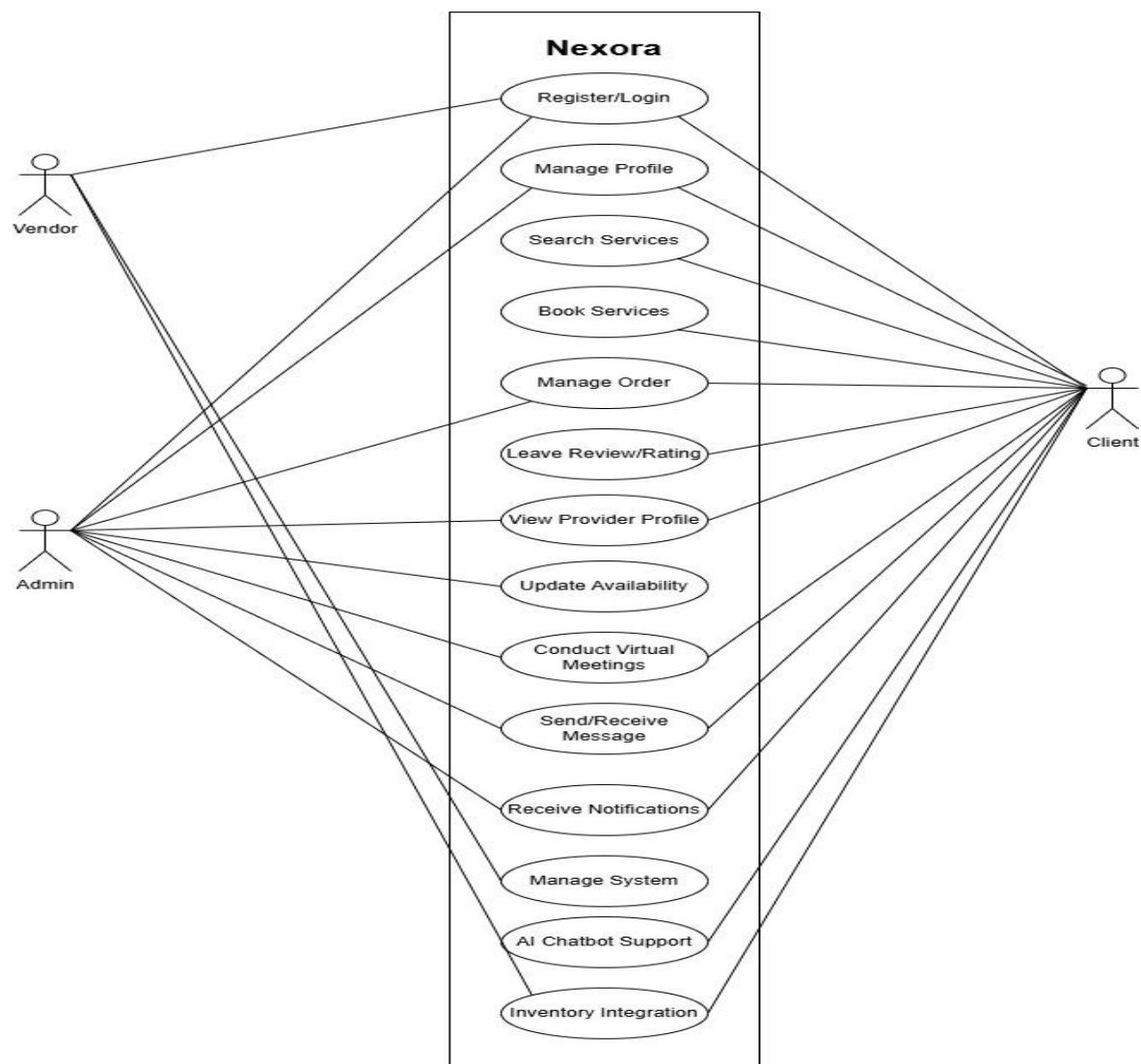


Figure 1 Use Case Diagram for Nexora

#### 3.2 Detailed Use Case

The table below indicates a comprehensive use case template filled in with an example drawn from the Online vehicle shipping management system.

*Table 2 Login*

|                  |                                                                                           |
|------------------|-------------------------------------------------------------------------------------------|
| Use case id      | 01                                                                                        |
| Use case name    | Login                                                                                     |
| Actors           | Admin, Employee/User                                                                      |
| Description      | Actor logs into Nexora system using valid credentials.                                    |
| Trigger          | User clicks login button.                                                                 |
| Pre-condition    | User must have an account.                                                                |
| Postcondition    | User is authenticated and redirected.                                                     |
| Normal flow      | 1. Open login page<br>2. Enter email/password<br>3. System validates<br>4. User logged in |
| Alternative flow | Invalid credentials → show error                                                          |
| Exceptions       | Network error, incorrect credentials                                                      |
| Business rules   | Internet is required                                                                      |

*Table 3: Get Registered*

|               |                                  |
|---------------|----------------------------------|
| Use case id   | 02                               |
| Use case name | User Registration                |
| Actors        | Admin, Employee/User, Client     |
| Description   | New user registers an account.   |
| Trigger       | User selects Register option.    |
| Pre-condition | User should not have an account. |

|                  |                                                                                            |
|------------------|--------------------------------------------------------------------------------------------|
| Postcondition    | Account created successfully.                                                              |
| Normal flow      | 1. Open registration page<br>2. Enter details<br>3. System validates<br>4. Account created |
| Alternative flow | Invalid data → re-enter                                                                    |
| Exceptions       | User already exists                                                                        |
| Business rules   | Valid email required                                                                       |

*Table 4: View System Modules*

|                  |                                                             |
|------------------|-------------------------------------------------------------|
| Use case id      | 03                                                          |
| Use case name    | View System Modules                                         |
| Actors           | Admin, User                                                 |
| Description      | User views modules like Invoice, Employees, Tasks, Payroll. |
| Trigger          | Dashboard loads.                                            |
| Pre-condition    | User is logged in.                                          |
| Postcondition    | Modules displayed.                                          |
| Normal flow      | 1. Login<br>2. Dashboard shows modules                      |
| Alternative flow | User not logged in                                          |
| Exceptions       | Server down                                                 |
| Business rules   | Role-based access                                           |

*Table 5: Company/Client Details*

|                  |                                                  |
|------------------|--------------------------------------------------|
| Use case id      | 04                                               |
| Use case name    | Enter Company/Client Details                     |
| Actors           | Admin                                            |
| Description      | Admin enters/upgrades company or client details. |
| Trigger          | Admin selects business settings.                 |
| Pre-condition    | Admin logged in.                                 |
| Postcondition    | Data stored.                                     |
| Normal flow      | 1. Open settings<br>2. Enter details<br>3. Save  |
| Alternative flow | Invalid/missing data                             |
| Exceptions       | Server down                                      |
| Business rules   | Only admin can update business details           |

*Table 6: View Item/Service Prices*

|               |                                                    |
|---------------|----------------------------------------------------|
| Use case id   | 05                                                 |
| Use case name | View Item/Service Prices                           |
| Actors        | Admin, User                                        |
| Description   | User views all product/service prices.             |
| Trigger       | User opens price list.                             |
| Pre-condition | User logged in.                                    |
| Postcondition | Price list displayed.                              |
| Normal flow   | 1. Login<br>2. Navigate to items<br>3. View prices |

|                  |                           |
|------------------|---------------------------|
| Alternative flow | No items available        |
| Exceptions       | Database error            |
| Business rules   | Only admin updates prices |

*Table 7: Create Invoice*

|                  |                                                                 |
|------------------|-----------------------------------------------------------------|
| Use case id      | 06                                                              |
| Use case name    | Create Invoice                                                  |
| Actors           | Admin                                                           |
| Description      | Admin generates invoice.                                        |
| Trigger          | Admin selects Create Invoice.                                   |
| Pre-condition    | Client and items exist.                                         |
| Postcondition    | Invoice saved.                                                  |
| Normal flow      | 1. Select Create Invoice<br>2. Enter details<br>3. Save invoice |
| Alternative flow | Missing details                                                 |
| Exceptions       | Server error                                                    |
| Business rules   | Only admin can create invoices                                  |

*Table 8: Mark Attendance (EMS Module)*

|               |                                |
|---------------|--------------------------------|
| Use case id   | 07                             |
| Use case name | Mark Attendance                |
| Actors        | Employee/User                  |
| Description   | User marks check-in/check-out. |



|                  |                                                     |
|------------------|-----------------------------------------------------|
| Trigger          | User selects attendance button.                     |
| Pre-condition    | User logged in.                                     |
| Postcondition    | Attendance stored in Firestore.                     |
| Normal flow      | 1. Login<br>2. Select attendance<br>3. Check-in/out |
| Alternative flow | Duplicate check-in                                  |
| Exceptions       | Firestore error                                     |
| Business rules   | One check-in per day                                |

*Table 9: Add Employee*

|                  |                                               |
|------------------|-----------------------------------------------|
| Use case id      | 08                                            |
| Use case name    | Add Employee                                  |
| Actors           | Admin                                         |
| Description      | Admin adds new employee.                      |
| Trigger          | Admin selects Add Employee.                   |
| Pre-condition    | Admin logged in.                              |
| Postcondition    | Employee added.                               |
| Normal flow      | 1. Open Employees<br>2. Enter data<br>3. Save |
| Alternative flow | Duplicate email                               |
| Exceptions       | Network down                                  |
| Business rules   | Only admin adds employees                     |

*Table 10:Assign Task*

|                  |                                             |
|------------------|---------------------------------------------|
| Use case id      | 09                                          |
| Use case name    | Assign Task                                 |
| Actors           | Admin                                       |
| Description      | Admin assigns task.                         |
| Trigger          | Admin selects Assign Task.                  |
| Pre-condition    | Employee exists.                            |
| Postcondition    | Task stored.                                |
| Normal flow      | 1. Open Tasks<br>2. Enter task<br>3. Assign |
| Alternative flow | Missing details                             |
| Exceptions       | Server error                                |
| Business rules   | Deadlines required                          |

*Table 11:Manage organizational roles*

|               |                               |
|---------------|-------------------------------|
| Use case id   | 10                            |
| Use case name | Process Payroll               |
| Actors        | Admin                         |
| Description   | Admin processes salary.       |
| Trigger       | Admin selects Payroll module. |
| Pre-condition | Attendance available.         |
| Postcondition | Payroll stored.               |

|                  |                                                       |
|------------------|-------------------------------------------------------|
| Normal flow      | 1. Fetch attendance<br>2. Calculate salary<br>3. Save |
| Alternative flow | Missing attendance                                    |
| Exceptions       | Calculation error                                     |
| Business rules   | Monthly payroll only                                  |

*Table 12: View Dashboard*

|                  |                                |
|------------------|--------------------------------|
| Use case id      | 11                             |
| Use case name    | View Dashboard                 |
| Actors           | Admin, User                    |
| Description      | User views summary dashboard.  |
| Trigger          | User logs in.                  |
| Pre-condition    | User exists.                   |
| Postcondition    | Dashboard displayed.           |
| Normal flow      | 1. Login<br>2. Dashboard loads |
| Alternative flow | No data                        |
| Exceptions       | Slow connection                |
| Business rules   | Role-based panels              |

*Table 13 Download Invoice Pdf*

|                  |                                      |
|------------------|--------------------------------------|
| Use case id      | 12                                   |
| Use case name    | Download Invoice PDF                 |
| Actors           | Admin                                |
| Description      | Admin downloads invoice PDF.         |
| Trigger          | Admin clicks download.               |
| Pre-condition    | Invoice exists.                      |
| Postcondition    | PDF downloaded.                      |
| Normal flow      | 1. Open invoice<br>2. Click download |
| Alternative flow | Missing invoice data                 |
| Exceptions       | PDF generation error                 |
| Business rules   | Only admin downloads invoices        |

### 3.3 Functional Requirements

Functional requirements describe the specific features, operations, and behaviors that a system must provide in order to fulfill its intended purpose. These requirements define what the system should do and outline the interactions between users and the system. They cover core modules, user actions, system responses, data processing steps, and workflow logic that enable the system to perform its essential tasks.

Typically, functional requirements are organized into modules such as user management, authentication, data processing, reporting, communication features, task or workflow handling, and administrative controls. Each module represents a set of actions that users can initiate and the system must support, including validations, data storage, information retrieval, and automated operations. These requirements ensure that the system delivers consistent behavior, supports all intended use cases, and aligns with the overall objectives defined during requirement analysis.

#### 1. User Module (Admin + Employee/User)

- Users can securely register and log in using authentication mechanisms (JWT-based Authentication).
- Users can manage their personal profiles (name, contact, role, etc.).

- Employees can mark attendance (Check-In/Check-Out) and view their attendance history.
- Employees can view their assigned tasks and update task status.
- Users can access their payroll information based on attendance logs.
- Users can view their invoices (if applicable) and check payment status.
- Users can receive notifications related to tasks, attendance, or admin announcements.

## **2. Invoice Management Module**

- Admin can create new invoices for clients.
- Admin can add, edit, or delete clients and service items.
- System automatically calculates invoice totals, taxes, discounts, and final amount.
- Admin can track invoice status (Paid, Pending, Overdue).
- Admin can download invoices as PDF files.
- System stores invoice records securely in MongoDB.
- Admin can generate invoice-based financial reports.

## **3. Employee Management Module**

- Admin can add, edit, or remove employees from the system.
- Admin can create employee roles (Admin / Employee).
- Admin can assign tasks to employees and track task progress.
- All employee activity (attendance, tasks, payroll) is managed through MongoDB.
- Admin can approve/reject leave requests (if implemented).
- Admin can monitor employee performance through the dashboard.

## **4. Organizational Admin Module**

- Admin can configure business settings such as company information, products/services, and pricing.
- Admin can manage employee accounts (add, remove, update).
- Admin has full access to dashboard analytics (revenue, attendance, tasks, payroll).
- Admin can define role-based access to modules and features.
- Admin can confirm or reject critical actions such as task completion or payroll finalization.

### **3.4 Non-Functional Requirements**

Non-Functional Requirements define the quality standards, performance expectations, and operational constraints that the Nexora Management Suite must fulfill to ensure a reliable and seamless user experience. While functional requirements explain what the system does, non-functional requirements describe how well the system performs those functions. These quality attributes determine the system's efficiency, scalability, usability, security, and overall stability across different environments.

For Nexora, non-functional requirements are essential because the platform handles sensitive financial records, employee data, cloud-based operations, and real-time workflows. The system

must therefore maintain consistent performance under varying workloads, protect confidential information through secure mechanisms, and provide an intuitive interface suitable for users with different technical skill levels. Additionally, Nexora must ensure smooth synchronization across its modules—such as invoicing, attendance, payroll, and task management—so that organizations can rely on it for uninterrupted daily operations.

These requirements collectively ensure that the Nexora Management Suite remains dependable, user-friendly, scalable, and secure, meeting both current organizational needs and future expansion demands.

## **1. Usability**

The Nexora Business Suite is designed for easy use by business owners, administrators, and employees. The system ensures:

- Clean and simple interface for invoice and employee workflows.
- Easy navigation through dashboard and modules.
- Minimal learning curve, allowing users to quickly understand the system.
- Responsive UI accessible from desktop and mobile devices.

## **2. Performance**

The system must remain fast and responsive to daily business operations. Nexora ensures:

- Quick loading of invoices, attendance logs, and tasks.
- Efficient real-time data syncing using Firebase and MongoDB.
- Ability to handle a large number of invoices, employees, and attendance records.
- Low latency when generating invoices, updating tasks, or processing payroll.

## **3. Security**

Security is crucial, as Nexora handles sensitive financial and employee data. The system includes:

- Secure login using JWT-based authentication tokens.
- Role-based access to protect administrative functions.
- Encrypted storage of invoice, employee, and order data in cloud databases.
- Restricted access to attendance and payroll data (Admin only).
- Secure APIs to prevent unauthorized access or data manipulation.
- Logged activity for audit purposes (attendance, task status changes, payroll generation).

## **4. Portability**

Nexora is a web-based system designed to run across multiple platforms. It ensures:

- Compatibility with Windows, macOS, and Linux systems.
- Smooth performance on all modern browsers (Chrome, Edge, Firefox).
- Responsive layouts for tablets and mobile devices.
- Cloud-based access so users can log in from any location.

## **5. Reliability**

The system ensures dependable and stable performance.

- All calculations (invoice totals, payroll) must be accurate.
- Real-time updates must be consistent across all modules.
- Error-handling ensures system stability in case of network failures.
- The system must maintain data integrity even under high usage.

# **Chapter 4**

## **Design And Architecture**



## 4 Design and Architecture

This chapter presents the overall design and architectural structure of the **Nexora Business Suite**. It includes the system architecture, data representation techniques, process flows, and the UML design models that define how the Invoice Management System and Employee Management System interact with each other. The chapter explains how different components such as the MERN backend, Firebase services, cloud databases, and frontend modules coordinate to deliver a seamless, scalable, and integrated business management platform.

### 4.1 System Architecture

The system architecture of Nexora follows a structured multi-layered design that integrates frontend applications, backend services, core business modules, and a centralized database into a single cohesive workflow. The presentation layer consists of web-based interfaces developed using React.js, along with supporting dashboards and client-side modules that allow users, employees, and administrators to interact with the system efficiently. These interfaces communicate with the backend through RESTful APIs, ensuring secure and reliable data exchange.

The backend layer is implemented using Node.js and Express.js, which is responsible for handling all application logic, request routing, authentication mechanisms, middleware validation, and session handling. This layer acts as the central control unit of the system, coordinating communication between different functional modules. Feature-specific components such as invoice management, employee and user management, order processing, reporting, analytics, chatbot services, and payment handling operate as independent modules within the backend but remain interconnected through shared services.

At the data layer, MongoDB is used as the primary database to store application data including users, employees, invoices, orders, and transactional records. The use of MongoDB enables flexible schema design, efficient handling of large datasets, and seamless scalability across all integrated software components of Nexora. This layered architecture ensures modularity, maintainability, and smooth integration between the different systems that form the complete Nexora ecosystem.

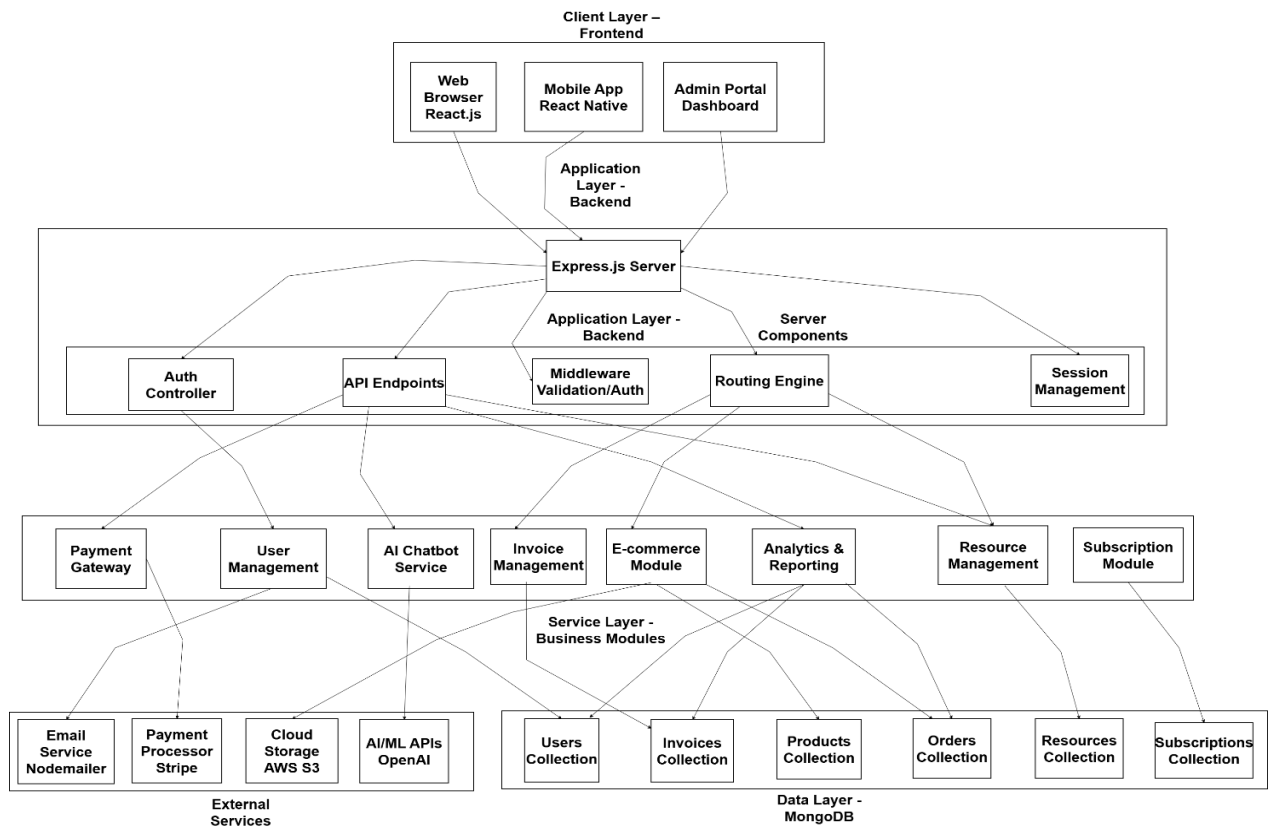


Figure 2 System Architecture

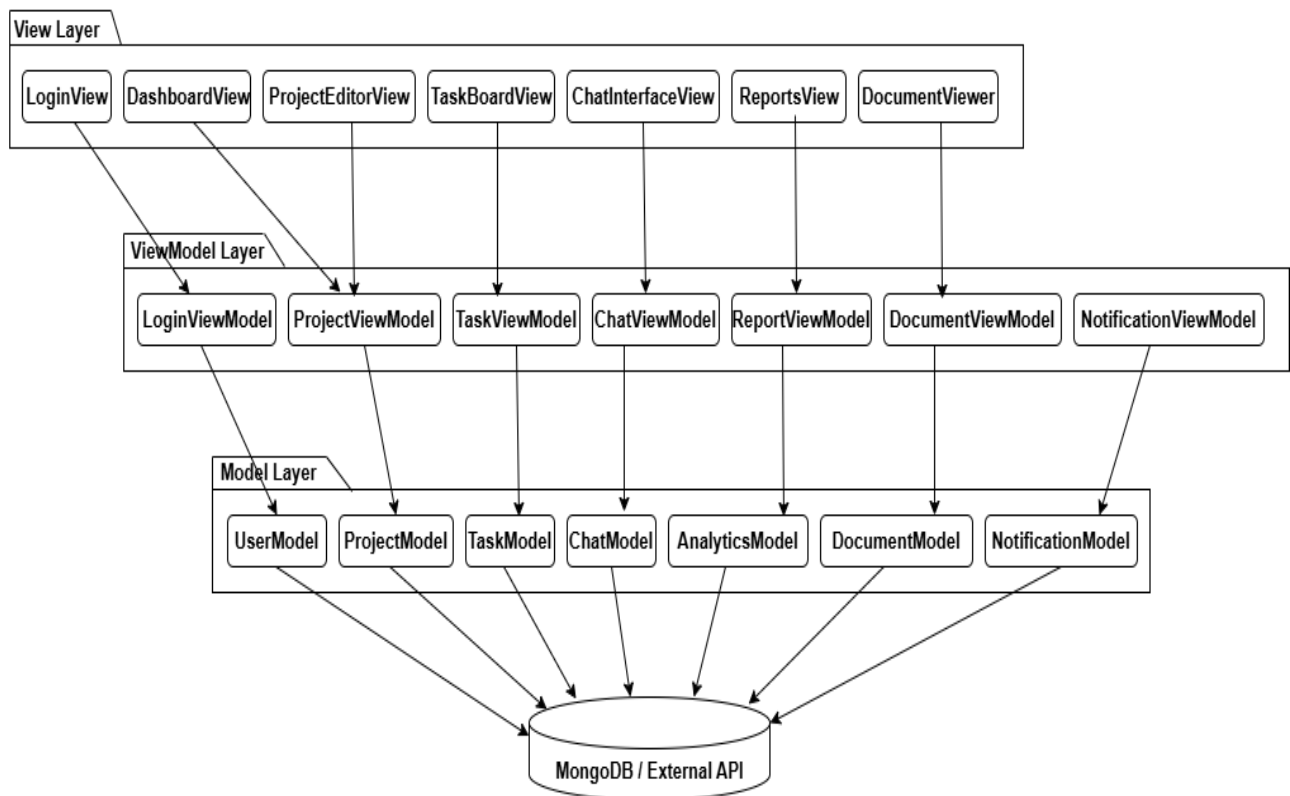


Figure 3 MVVM Architecture

The MVVM (Model–View–ViewModel) architecture adopted in Nexora provides a clear separation between user interface, application logic, and data management layers. The View layer consists of multiple user-facing screens such as LoginView, DashboardView, TaskBoardView, ChatInterfaceView, ReportsView, and DocumentViewer, which are responsible only for displaying data and capturing user interactions. These views do not directly communicate with the database; instead, they forward user actions to the ViewModel layer.

The ViewModel layer acts as an intermediary, containing business logic and handling operations such as validation, data processing, and request coordination. ViewModels like TaskViewModel, ProjectViewModel, ChatViewModel, and ReportViewModel interact with the underlying data models and ensure that only processed and relevant data is sent back to the views.

The Model layer represents the core data entities of Nexora, including users, projects, tasks, chats, analytics records, documents, and notifications. These models are mapped to MongoDB collections and external APIs, enabling persistent storage and retrieval of data. By implementing MVVM, Nexora achieves improved modularity, scalability, and maintainability, while allowing frontend and backend components to evolve independently without impacting overall system stability.

In addition, the MVVM architecture in Nexora enhances testability and long-term scalability of the system by decoupling UI logic from core business operations. Since ViewModels do not directly depend on the View layer, changes in user interface design—such as adding new dashboards or modifying existing screens—do not impact backend logic or data models. This architectural flexibility is particularly important for Nexora, as the platform integrates multiple subsystems including employee management, invoicing, analytics, and AI-driven communication services.

## **Data Representation**

In Nexora, system data is represented using structured and schema-based models to accurately reflect real-world business entities and workflows. Core domains such as users, employees, attendance records, tasks, invoices, clients, payroll details, orders, and role-based permissions are each mapped to dedicated MongoDB collections. Every entity is defined with clearly structured attributes, relationships, and constraints to ensure data consistency across the system.

The Employee Management System represents entities like employees, attendance logs, assigned tasks, reports, and payroll records as document-based models stored in MongoDB, allowing flexible yet reliable data handling. Similarly, the Invoice and Pet Store modules use MongoDB schemas to manage clients, invoices, products, pets, orders, and transaction records. This unified data representation enables efficient CRUD operations, supports complex queries, and ensures seamless integration between different software modules.

By using a centralized MongoDB-based data layer, Nexora maintains a consistent source of truth across all subsystems, simplifies backend development, and ensures that frontend components receive accurate and up-to-date information for decision-making and reporting purposes. MongoDB's document-oriented data representation aligns well with Nexora's modular system design, allowing each software component to store and manage its data

independently while still sharing a common database infrastructure. Flexible schemas enable the system to evolve over time by accommodating new fields—such as additional invoice attributes, expanded employee records, or enhanced pet and order details—without disrupting existing data.

## 4.2 Data Dictionary

Below is the data dictionary table

### a. Data Dictionary

**Table 14: Data Dictionary**

| Entity       | Attribute                                    | Type                           | Description                                                             |
|--------------|----------------------------------------------|--------------------------------|-------------------------------------------------------------------------|
| User         | userId, name, email, password, role          | ObjectId, String               | loginUser()<br>registerUser()<br>updateProfile()<br>authenticateRole()  |
| Subscription | subscriptionId, type, startDate, expiryDate  | ObjectId, String, Date         | activatePlan()<br>renewPlan()<br>checkSubscriptionStatus()              |
| Invoice      | invoiceId, amount, dueDate, status           | ObjectId, Number, Date, String | createInvoice()<br>updateInvoice()<br>calculateTotal()<br>generatePDF() |
| Product      | productId, name, price, stock                | ObjectId, String, Number       | addProduct()<br>updateProduct()<br>checkStock()                         |
| Order        | orderId, productId, quantity, status         | ObjectId, Number, String       | placeOrder()<br>updateOrderStatus()<br>calculateOrderAmount()           |
| Resource     | resourceId, name, availability, taskAssigned | ObjectId, String, Boolean      | assignTask()<br>updateAvailability()<br>trackResourceStatus()           |
| Report       | reportId, type, generatedBy                  | ObjectId, String               | generateReport()<br>filterReports()<br>exportReport()                   |

## 4.3 Process Flow / Representation

The activity diagram outlines the step-by-step operations within the Nexora system. It starts with user registration or login, followed by accessing the dashboard. From there, users can navigate to various modules such as invoicing, resource management, or the e-commerce store. Each module allows specific operations like viewing services, entering business details, checking prices, booking services, and interacting with the AI chatbot. The diagram also

highlights decision points, such as authentication verification and service availability, ensuring logical flow and system validation at each step. This visual representation simplifies understanding how users interact with the system and how data flows across different components.

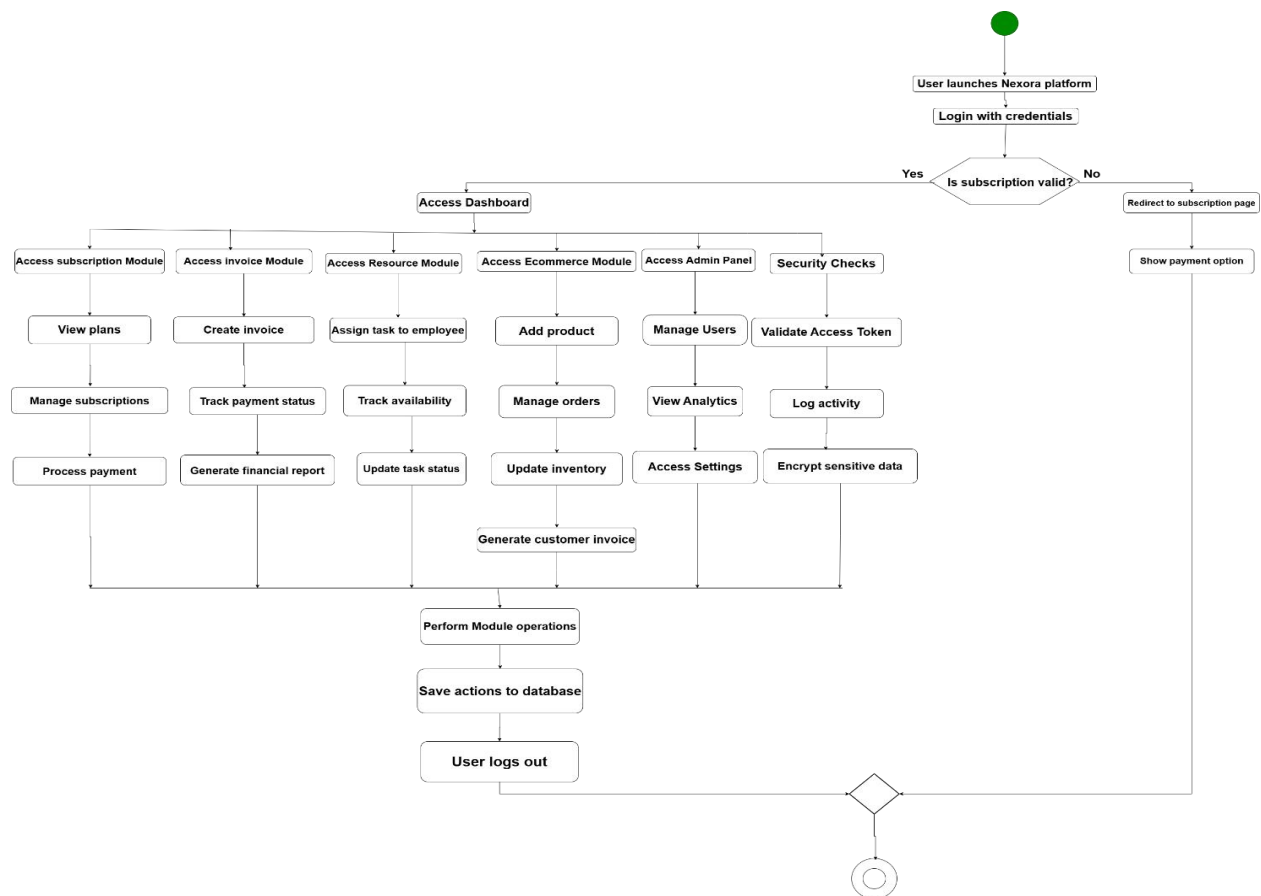


Figure 4 Activity Diagram

## 4.4 Design Models

Design models are visual and structural representations that describe how different components of a software system are organized, interact, and function before actual implementation. These models help developers and stakeholders clearly understand the architecture, workflows, and internal logic of the system. They also ensure that the final software aligns with the documented requirements and follows a consistent, maintainable design approach.

### 4.4.1 Class Diagram

The class diagram illustrates the structural design of the system by defining its core entities and their relationships. It shows how components such as User, Admin Panel, Subscription, Orders, Resources, Invoice, Security, and Chatbot interact with each other through associations and shared responsibilities. Each class represents a key functional module, while their connections highlight data flow and system dependencies. The diagram also reflects important operations such as stock updates, access control, and resource handling, providing a clear overview of how different modules collaborate within the system architecture.

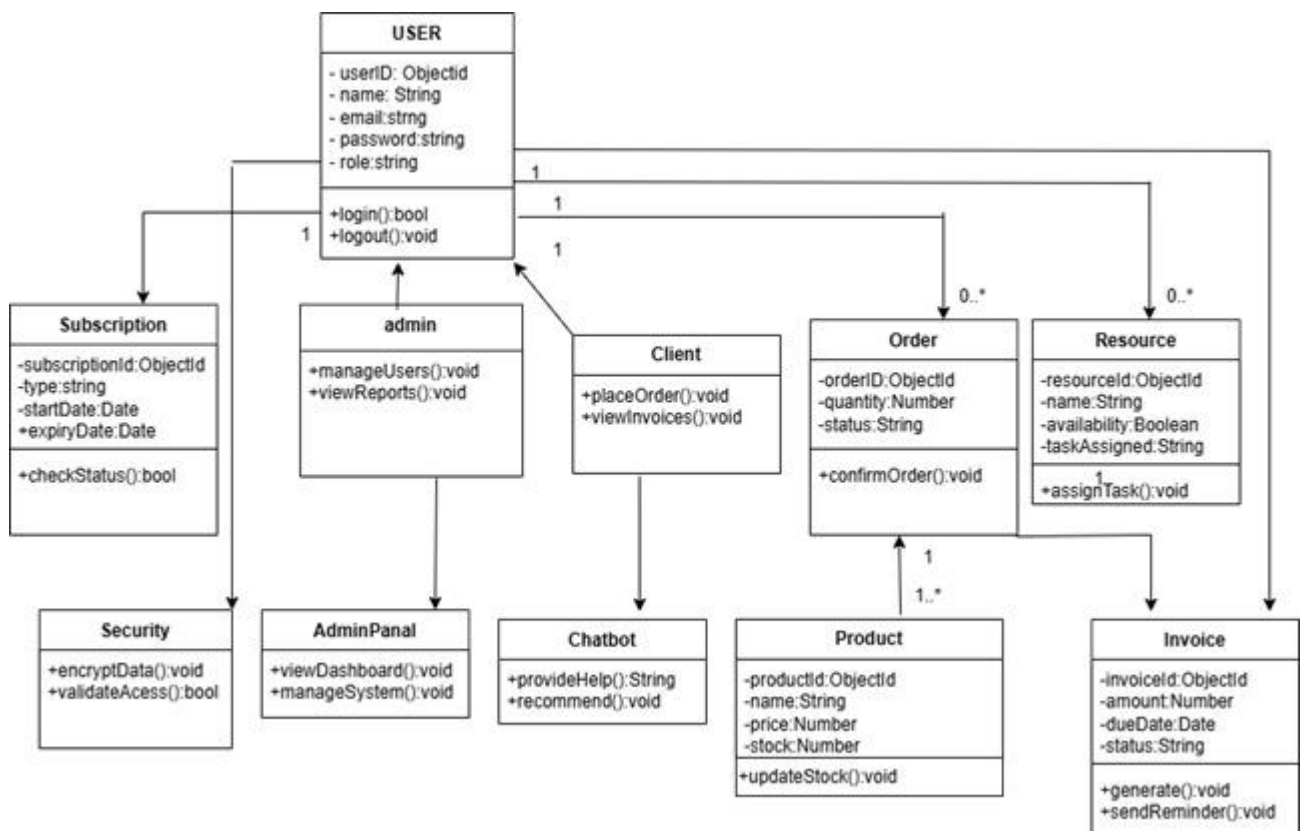


Figure 5 Class Diagram

#### 4.4.2 ERD Diagram

This Entity-Relationship Diagram represents the core database structure of the system, showing how users, orders, invoices, subscriptions, products, and reports are interconnected. The User entity is central and links to multiple modules, including subscriptions, orders, and reports. Orders contain multiple products through the Order\_Line entity, which tracks product quantity and details. Invoices are generated based on user orders, maintaining financial records such as amount and due dates. The Resource and Report entities support task assignment and performance tracking, while Subscription manages user access levels. Overall, the ERD illustrates a well-organized relational flow between business operations and transactional data.

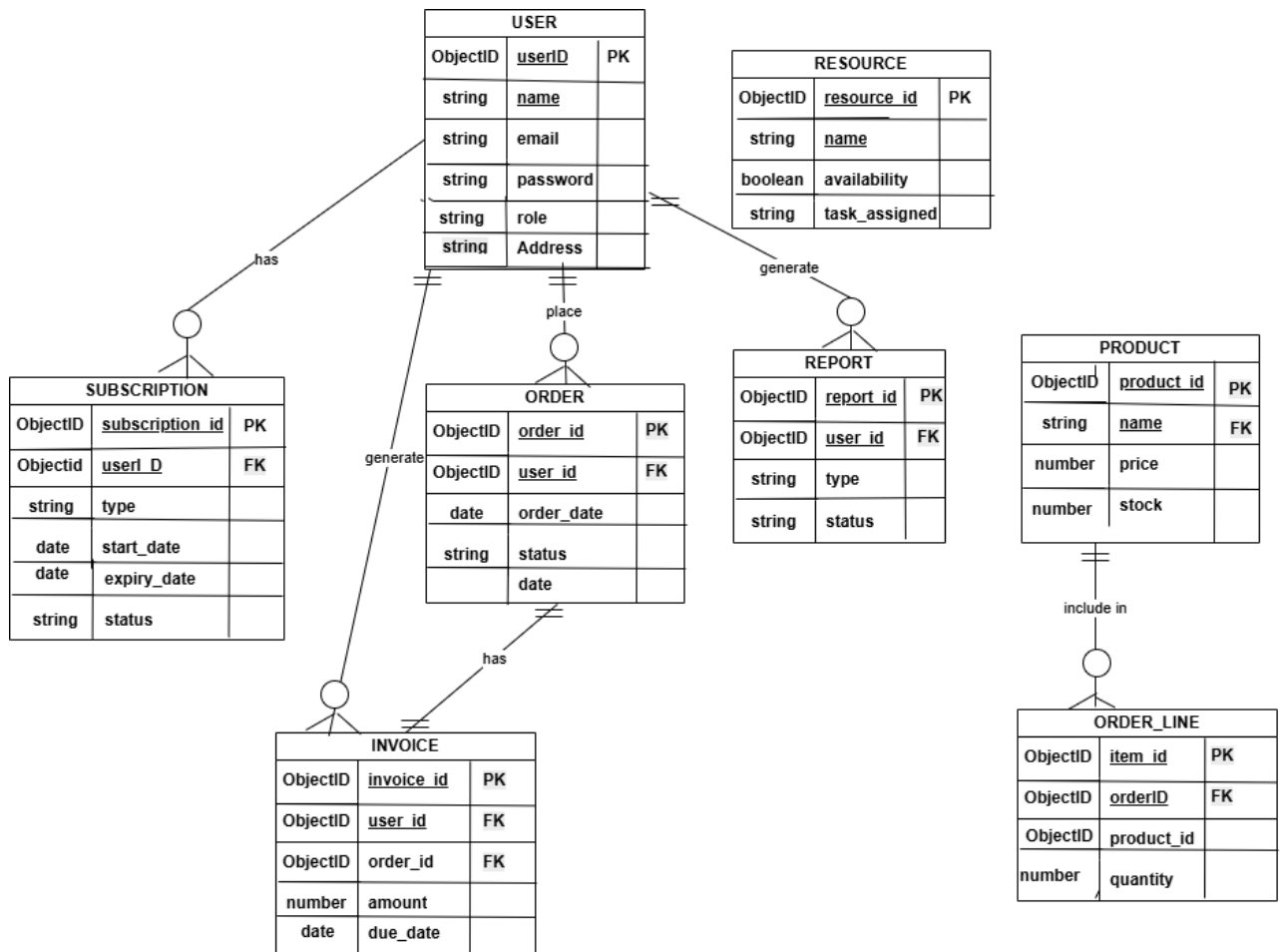


Figure 6 ERD Diagram

### 4.4.3 Sequence Diagram

The sequence diagram of the Nexora system illustrates the real-time interaction between different system components, showing how user actions are processed from initiation to response. It begins with the user performing an action, such as logging in, booking a service, or submitting a business profile. This request is captured by the View (frontend interface) and passed to the ViewModel, which acts as a mediator between the UI and the core logic. The ViewModel processes the input, performs any necessary validations, and forwards the request to the Model, which contains the business logic. The Model then interacts with the MongoDB database to retrieve or update the required information. Once the data operation is complete, the result is returned back through the Model and ViewModel to the View, where the user receives a response, such as confirmation messages or requested data. The diagram maps this interaction using lifelines and arrows that represent function calls, data retrieval, and responses in sequential order. It highlights the system's flow of control and data during key operations. This helps developers understand how different layers of the MERN-based architecture communicate effectively. The sequence diagram also ensures smooth system behavior and assists in identifying potential performance bottlenecks or design flaws. Overall, it is a vital tool for visualizing the dynamic behavior and responsiveness of the Nexora web application.

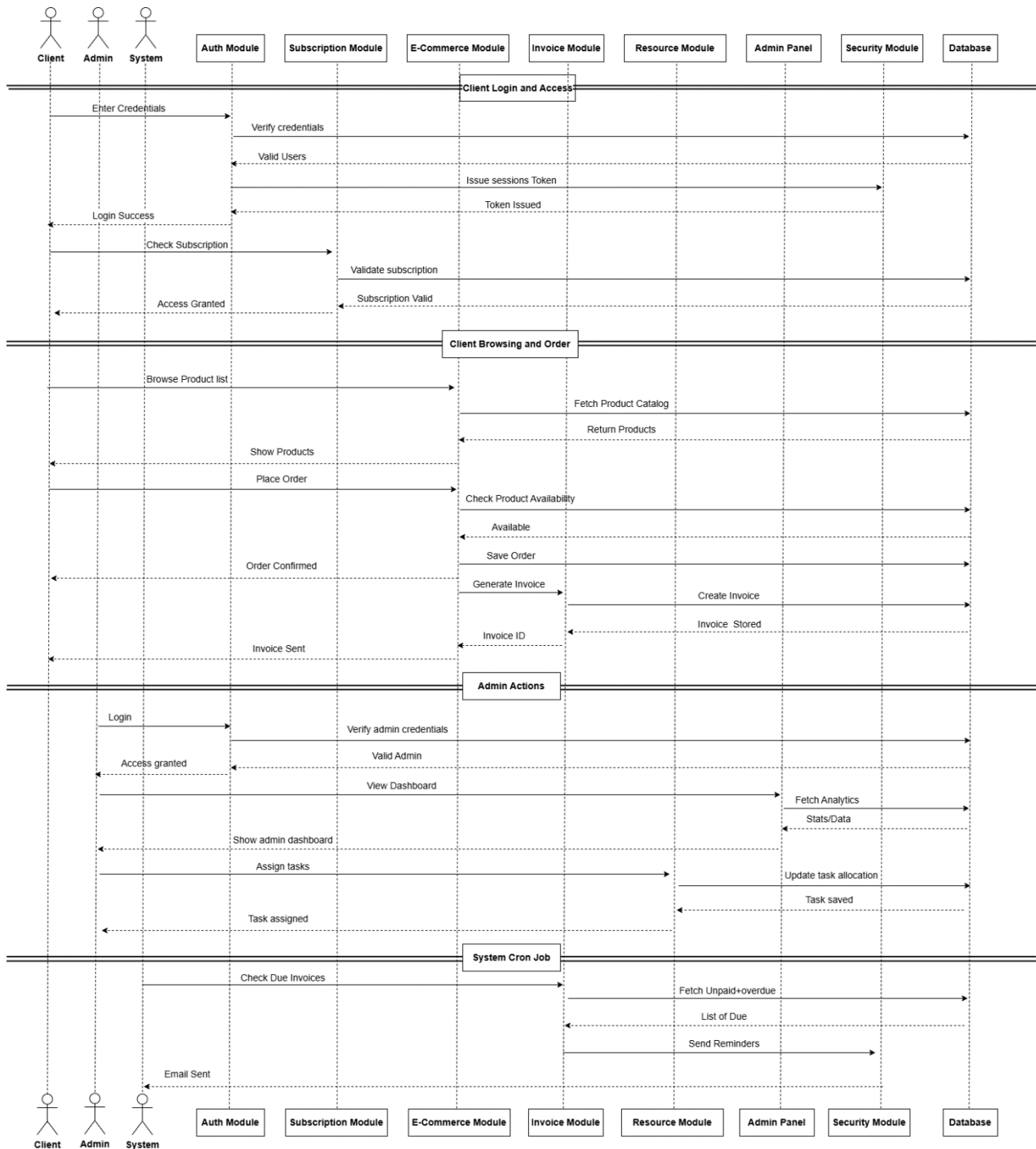


Figure 7 Sequence Diagram

#### 4.4.4 State Transition Diagram

Design models are visual and structural representations that describe how different components of a software system are organized, interact, and function before actual implementation. These models help developers and stakeholders clearly understand the architecture, workflows, and internal logic of the system. They also ensure that the final software aligns with the documented requirements and follows a consistent, maintainable design approach.



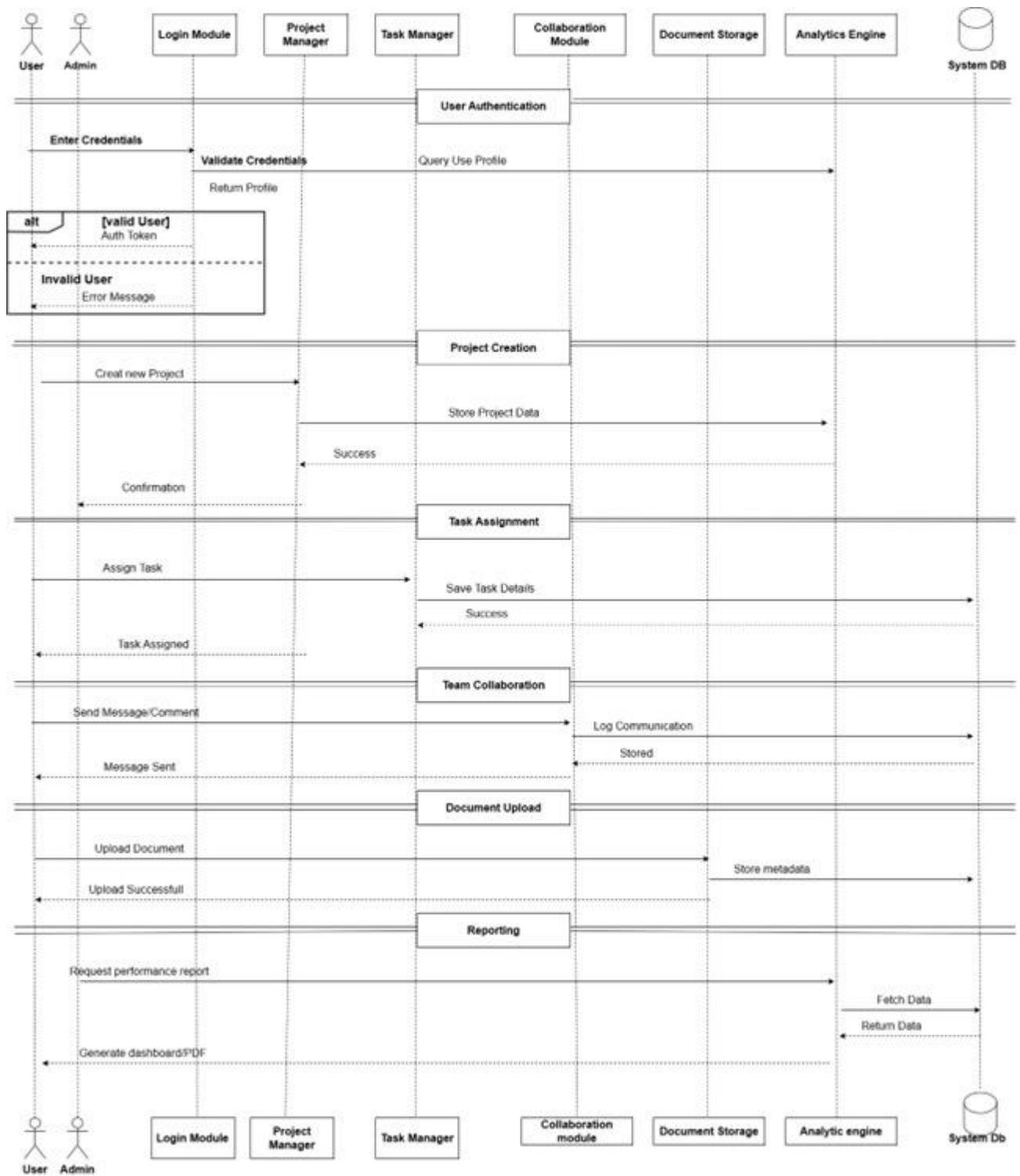


Figure 8 State Transition Diagram

## 4.4.5 Data Flow Diagram

### DFD Level 0

The Level 0 Data Flow Diagram provides a high-level overview of Nexora, treating the entire system as one unified process that interacts with external users such as Admins, Employees, and

Clients. At this level, the internal working of the system is not shown; instead, the focus is on how data enters the system and what outputs are generated.

Users provide inputs like login credentials, invoice information, attendance entries, task requests, and business details. Nexora processes these inputs and returns appropriate outputs, including dashboard access, employee records, task updates, invoice data, and system notifications. This diagram highlights the system's core purpose — receiving user requests, processing them, and returning meaningful results.

Overall, the Level 0 DFD represents Nexora as a centralized business management platform that handles all user interactions at the most fundamental level, without detailing the internal modules or data handling steps.

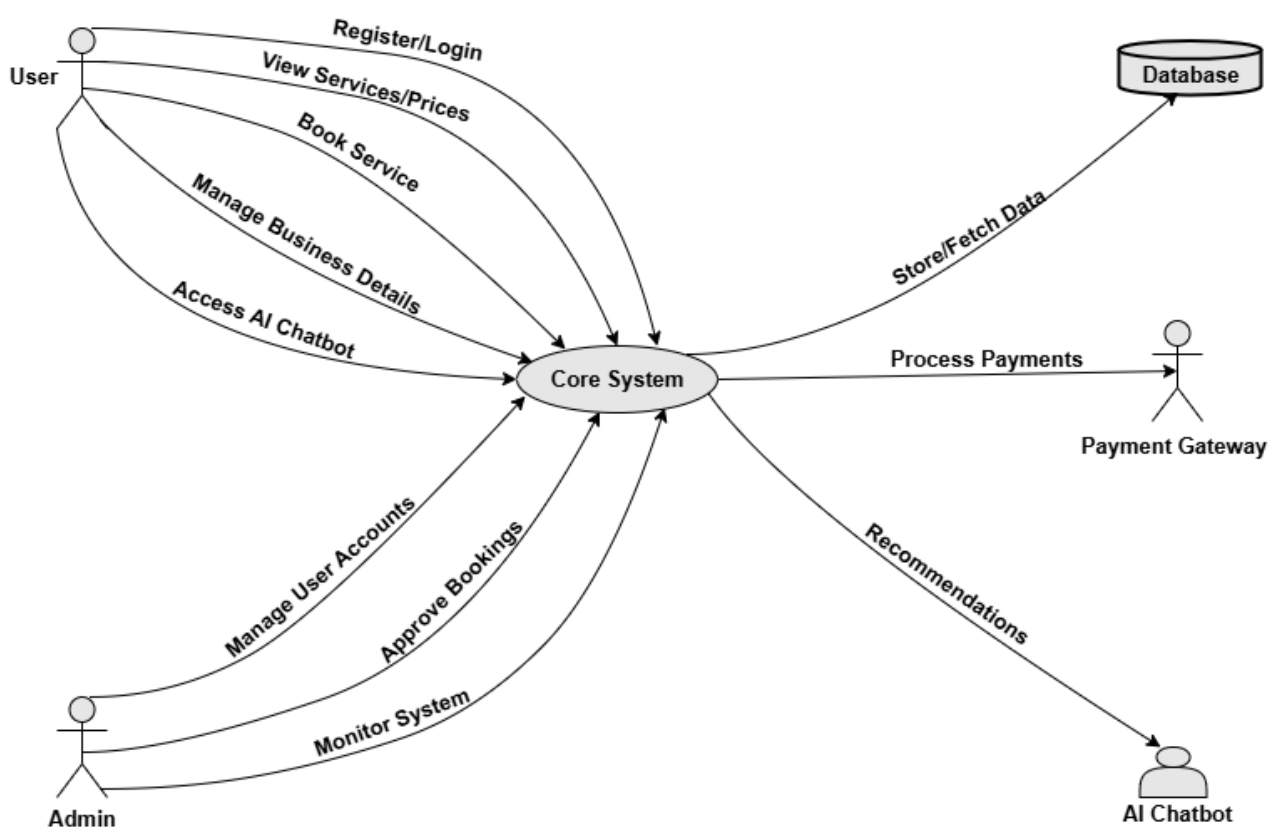


Figure 9 DFD level 0

## DFD Level 1

The Level 1 Data Flow Diagram breaks down Nexora into its major functional sub-processes, providing a clearer view of how different modules operate within the system. These key processes include User Management, Order Handling, Resource Allocation, Invoice Management, and other supportive features. Each sub-process exchanges information with a central database, ensuring consistent and synchronized data across the system.

User Management handles tasks like registration and login, and then forwards verified user details to other modules such as Orders or Subscription. Order Handling communicates with Product and

Invoice modules to process selections, calculate totals, and generate billing details. Resource Allocation manages employee or asset assignments, while additional modules—such as Chatbot Support, if applicable—retrieve recommendations and stored knowledge from the system database.

Overall, the Level 1 DFD provides a more detailed view of Nexora’s functional structure, showing how the main subsystems collaborate through shared data flows to support the platform’s business operations.

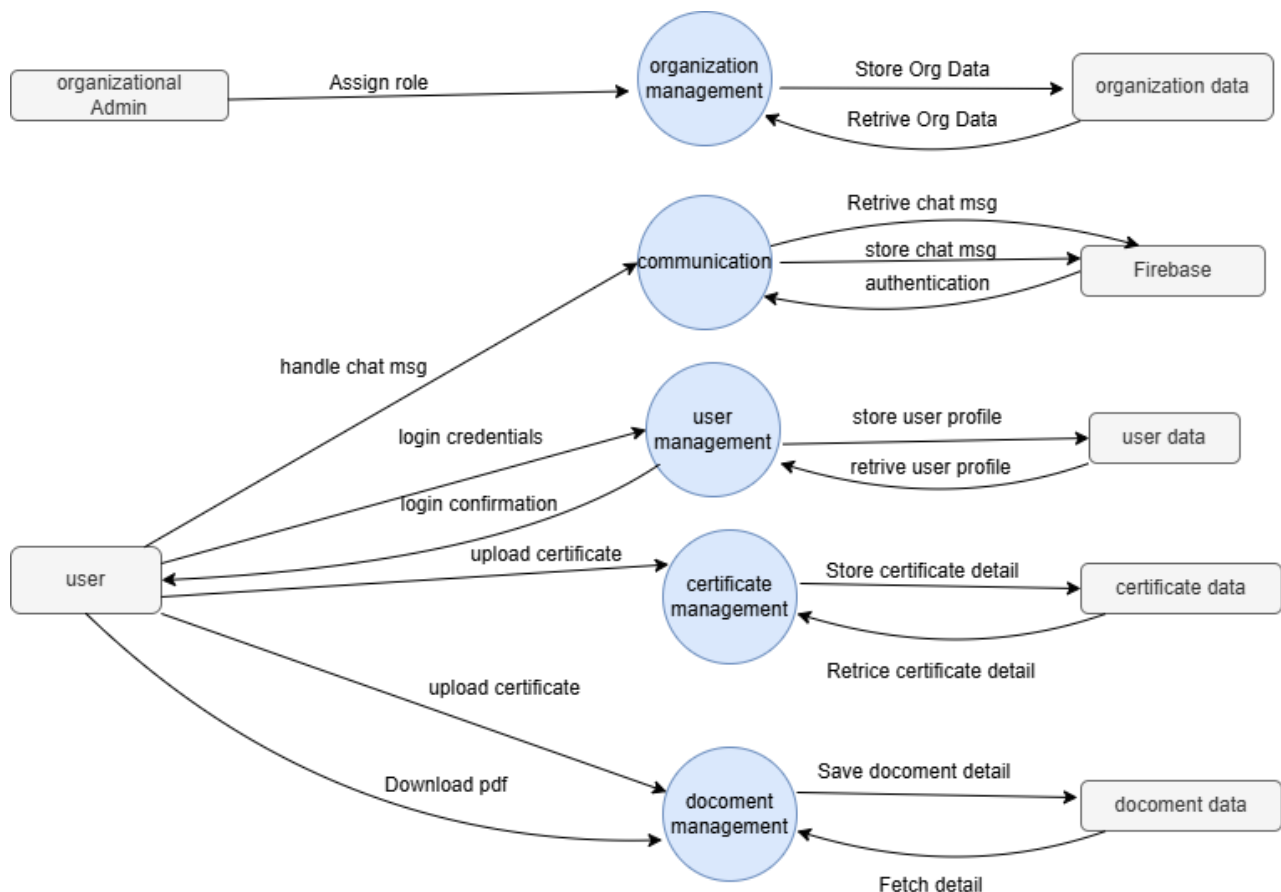


Figure 10 DFD Level 1

## DFD Level 2

The Level 2 Data Flow Diagram provides a deeper breakdown of Nexora’s internal workflows and shows how each sub-process performs detailed operations. Within the User Management module, registration and login requests are validated, user roles are assigned, and profile changes are securely stored. Order Handling verifies product availability, processes order confirmations, and updates inventory records in real time.

The Invoice Management process generates invoices, calculates totals with applied pricing rules, assigns due dates, and may trigger automated payment reminders. The Subscription Module continuously checks the user’s subscription validity and restricts access to premium features if the plan expires. Additionally, if an AI Chatbot or recommendation component exists, it receives queries, analyzes user intent, and retrieves appropriate responses from stored data.

All these internal processes rely on a shared, centralized database to maintain data integrity, provide secure record-keeping, and ensure smooth communication between Nexora’s functional modules.

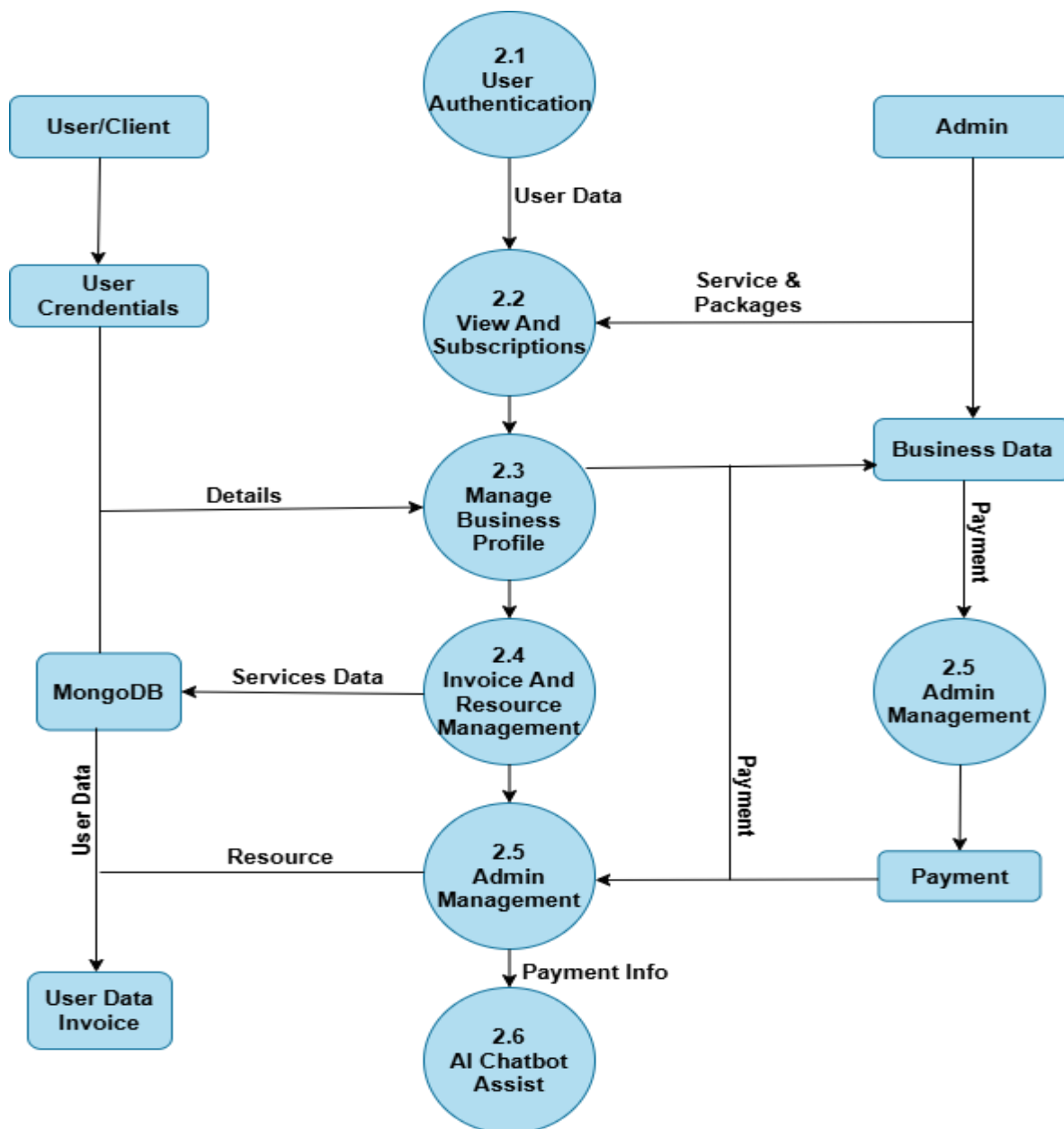


Figure 11 DFD Level 2

# **Chapter 5**

## **Implementation**

## 5 Implementation

The implementation phase focuses on converting the system design into a fully functional web-based business management platform using the MERN stack across all major system modules. Nexora is developed using React JS for the frontend, Node.js and Express for backend APIs, and MongoDB as the centralized database for managing invoices, employee records, attendance logs, tasks, payroll data, and pet store orders. Authentication and secure access are handled through JWT-based mechanisms integrated within the backend services.

The implementation follows a modular approach, where each feature is developed as an independent component or service to ensure maintainability, scalability, and ease of debugging. Nexora also uses a layered architecture common to modern web applications, dividing the system into:

- Frontend Layer (React Components / UI Layer)
- Backend API Layer (Express Controllers & Routes)
- Database Layer (MongoDB collection)

The implementation includes the following major modules:

### 1. User Authentication Module

Handles login, registration, role-based access, and secure session management using JWT authentication integrated with the Node.js and Express backend.

### 2. Invoice Management Module

Responsible for creating invoices, calculating totals, managing clients, storing invoice data in MongoDB, and generating downloadable PDF files.

### 3. Client & Product Module

Manages client records, service items, pricing, product listings, and their integration with invoice creation.

### 4. Employee Management Module

Allows admin to add, remove, and update employee profiles stored in MongoDB.

### 5. Attendance Tracking Module

Implements real-time Check-In/Check-Out logging using MongoDB collections.

### 6. Task Assignment Module

Enables admins to assign, update, and monitor employee tasks with real-time updates.

### 7. Payroll Management Module

Automatically calculates salaries based on attendance, stores payroll logs, and provides monthly summaries.

### 8. Dashboard & Analytics Module

Displays visual stats for invoices, attendance, tasks, and payroll through dynamic data fetching from MongoDB.

## 5.1 Algorithms

### 1. User Authentication Algorithm

User enters email & password

- System checks:  
if (userExistsInDatabase) → verifyCredentials()  
else → showError("Account not found")
- On success:  
generateJWT() → fetchUserRole() → redirectToDashboard()
- On failure:  
showError(retryLogin)

### 2. Invoice Generation Algorithm (Invoice Module – MERN)

Admin selects Create Invoice

- System loads product/client data from MongoDB
- Admin enters: items, quantity, price, discount
- System:
  - a. calculateSubtotal()
  - b. applyDiscount()
  - c. generateTotalAmount()
    - Save invoice in MongoDB
    - Generate downloadable PDF → attach invoiceId
    - Update invoiceHistory()

### 3. Attendance Check-In/Out Algorithm

Employee clicks Check-In or Check-Out

- System checks previous day status:  
if (alreadyCheckedIn) → blockDuplicate()
  - Capture timestamp → storeAttendance(MongoDB)
  - Update attendanceStatus = “Checked-In / Checked-Out”
  - Push update to admin dashboard

### 4. Task Assignment & Tracking Algorithm

Admin selects employee → createTask()

- System stores task details (title, description, deadline) in MongoDB
- Employee receives notification
- Employee updates:
  - a. markInProgress()
  - b. markCompleted()
    - System updates task status → updateDashboard()
    - Admin reviews task logs

### 5. Payroll Calculation Algorithm

System loads attendance logs from MongoDB

- For each employee:
  - a. countWorkingDays()
  - b.  $\text{calculateSalary} = \text{baseSalary} \times \text{daysPresent}$
  - c. applyDeductions()
    - Store payroll in MongoDB
    - Notify employee: salaryProcessed()

## **6. Client & Product Management Algorithm (Invoice Module)**

**Client Workflow:**

- Admin selects Add Client
- Enter: name, company, email, phone
- System validates  $\rightarrow$  saveClient(MongoDB)

**Product Workflow:**

- Admin adds product/service: name, price, category
- System validates  $\rightarrow$  saveProduct(MongoDB)
- On invoice creation  $\rightarrow$  fetchProductList()

## **7. Role-Based Access Control Algorithm**

System assigns roles:

- Admin  $\rightarrow$  full access
- Employee  $\rightarrow$  attendance/tasks/payroll view

For every request:

- checkUserRole()
- if roleAllowed(action)  $\rightarrow$  grantAccess()
- else  $\rightarrow$  denyAccess()
- Route protection enforced at frontend & backend

## **8. Real-Time Dashboard Update Algorithm**

Any system changes triggers event:

- a. newInvoice
- b. attendanceUpdate
- c. newTask / completedTask
- d. newOrder
  - Backend updates database state
  - Dashboard fetches updated metrics via APIs
  - Dashboard displays:
    - a. total employees
    - b. total invoices (paid/unpaid)
    - c. attendance summary
    - d. task status metrics



## 5.2 External APIs

This section provides an overview of all external APIs integrated into the Nexora Business Suite, explaining their purpose, functionality, and how they support different parts of the system. These APIs form the backbone of the platform by enabling core features such as secure authentication, employee data management, invoice processing, cloud storage, task handling, attendance tracking.

Through the MERN-based backend APIs and cloud-hosted MongoDB Atlas database, Nexora achieves seamless communication between modules, real-time data consistency, and scalable operations. The APIs collectively ensure secure data exchange, modular interaction, and reliable system performance across all integrated business modules.

Together, these APIs enable Nexora to operate as a secure, efficient, and fully integrated business management solution.

**Table 15: External APIs**

| <b>Name of API</b>                | <b>Description of API</b>                                               | <b>Purpose of Usage</b>                                                    | <b>Function/Class Name where Used</b>                            |
|-----------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>JWT Authentication API</b>     | Token-based authentication mechanism for secure user sessions           | To authenticate users and protect backend routes                           | AuthMiddleware.verifyToken(), generateJWT()                      |
| <b>MongoDB Atlas API</b>          | Cloud-hosted NoSQL database service                                     | To store employees, invoices, attendance, payroll, tasks, and orders       | EmployeeController, InvoiceController, OrderController           |
| <b>Node.js / Express REST API</b> | Backend API layer for business logic and data handling                  | To process CRUD operations and system workflows                            | invoiceRoutes, employeeRoutes, orderRoutes                       |
| <b>PDF Generation API</b>         | JavaScript-based PDF generation library                                 | To generate downloadable invoice PDFs                                      | PDFService.generateInvoicePDF()                                  |
| <b>JWT Authentication API</b>     | JSON Web Token-based authentication for securing Express API endpoints. | To verify user roles (Admin / Employee) and protect backend routes.        | AuthMiddleware.verifyToken(), generateJWT()                      |
| <b>pdf-lib / jsPDF API</b>        | JavaScript PDF generation & editing library.                            | To generate invoice PDFs, calculate totals, and export downloadable files. | PDFService.generateInvoicePDF(), InvoiceController.downloadPDF() |

## 5.3 User Interface

The Nexora Business Suite offers a clean and user-friendly interface. The React-based Invoice system provides simple navigation for managing clients, products, and invoices, while the MongoDB-powered Employee System allows quick access to attendance, tasks, and payroll. The overall UI is clear, responsive, and easy for both admins and employees to use.

### Mockups:

#### Splash Screen (Nexora Invoice)

The splash screen provides a welcoming introduction to the Nexora Invoice System, guiding users toward the main application. It features bold typography and clear navigation options for both existing and new users. The design ensures a smooth transition into the system while maintaining a modern and visually appealing interface that reflects the branding of Nexora.

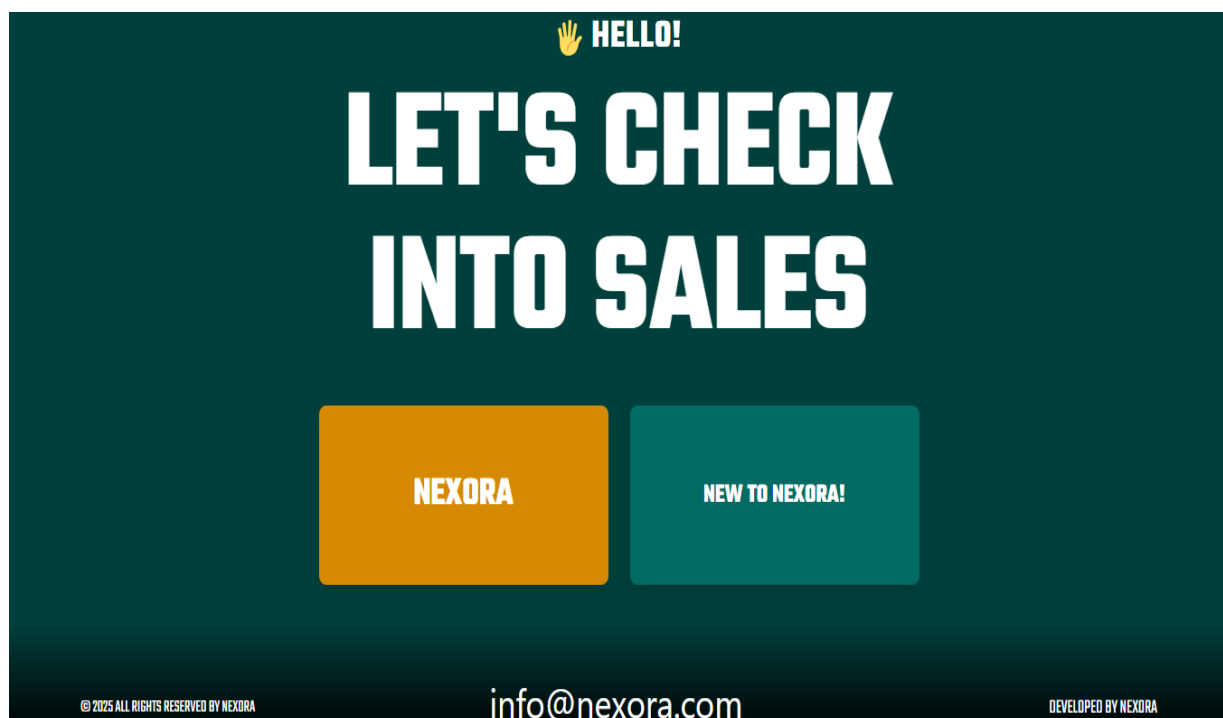


Figure 12 Splash Screen

### Login Screen (Nexora Invoice)

The login screen provides a clean and minimal interface where users can securely access their Nexora account. It includes fields for company name, email, and password, ensuring authenticated entry into the system. The simple layout and clear typography make the login process fast, intuitive, and user-friendly, helping users quickly move into the main dashboard.

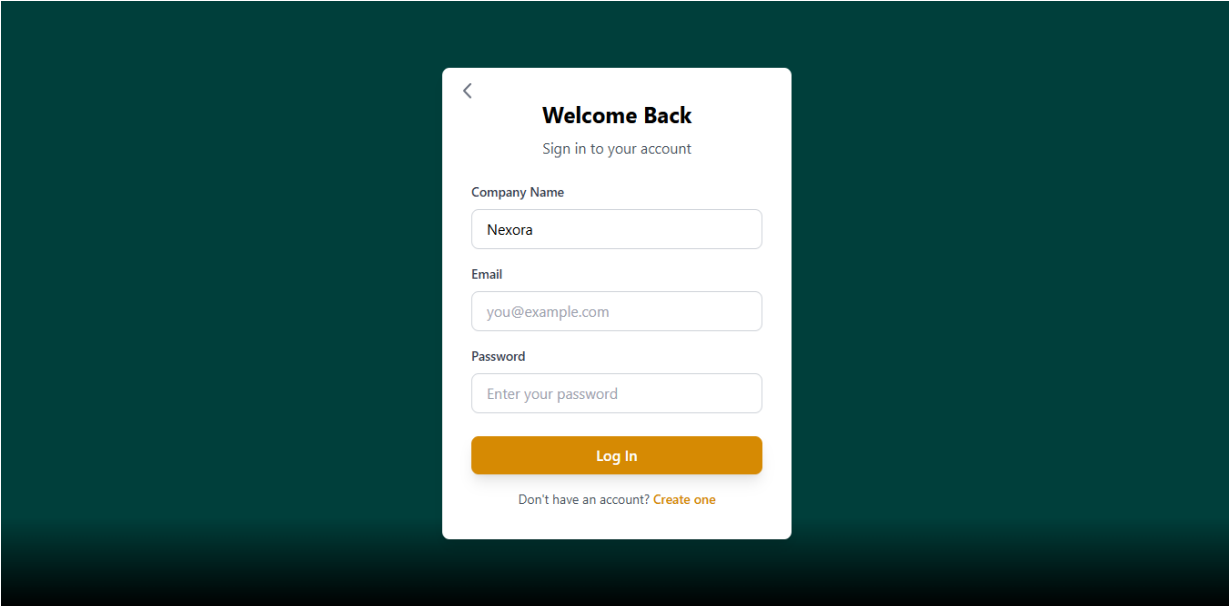


Figure 13 Login Screen

### Sign up Screen (Nexora Invoice)

The registration screen enables new users to create their Nexora account by providing essential details such as full name, company name, email, and password. Its clean and structured layout ensures a smooth onboarding experience, allowing businesses to quickly set up their profile and start using the invoicing system. Clear input fields and guided prompts make the registration process simple and user-friendly.

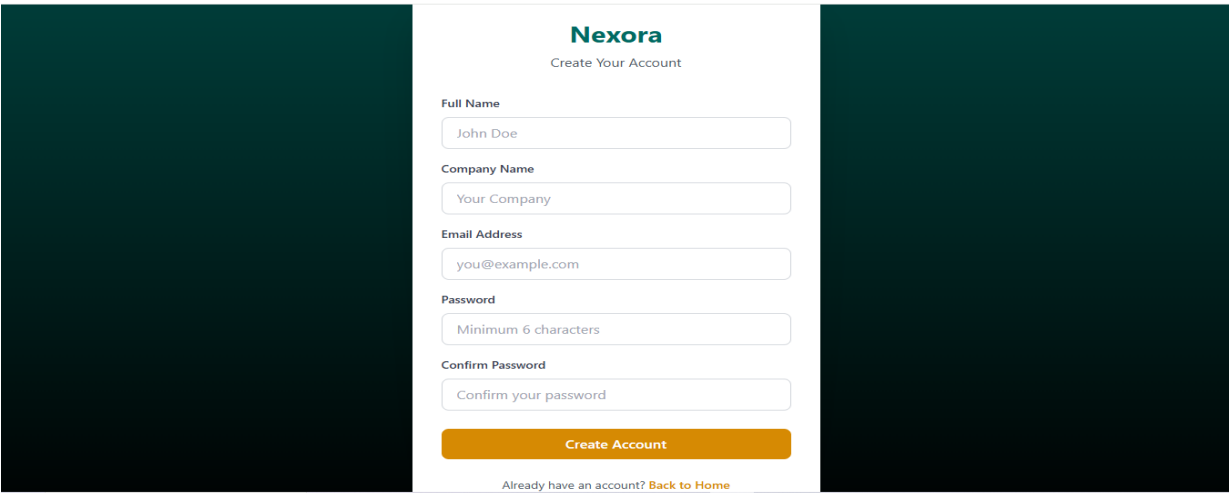


Figure 14 Sign Up Screen

Dashboard / Home Screen (Nexora Invoice)

The dashboard provides a quick overview of key business metrics, including total invoices, payments received, and pending amounts. With a clean layout and date filters, users can instantly analyze financial activity within a selected period. The left-side navigation menu allows easy access to all modules, making the dashboard the central hub for monitoring and managing business operations efficiently.

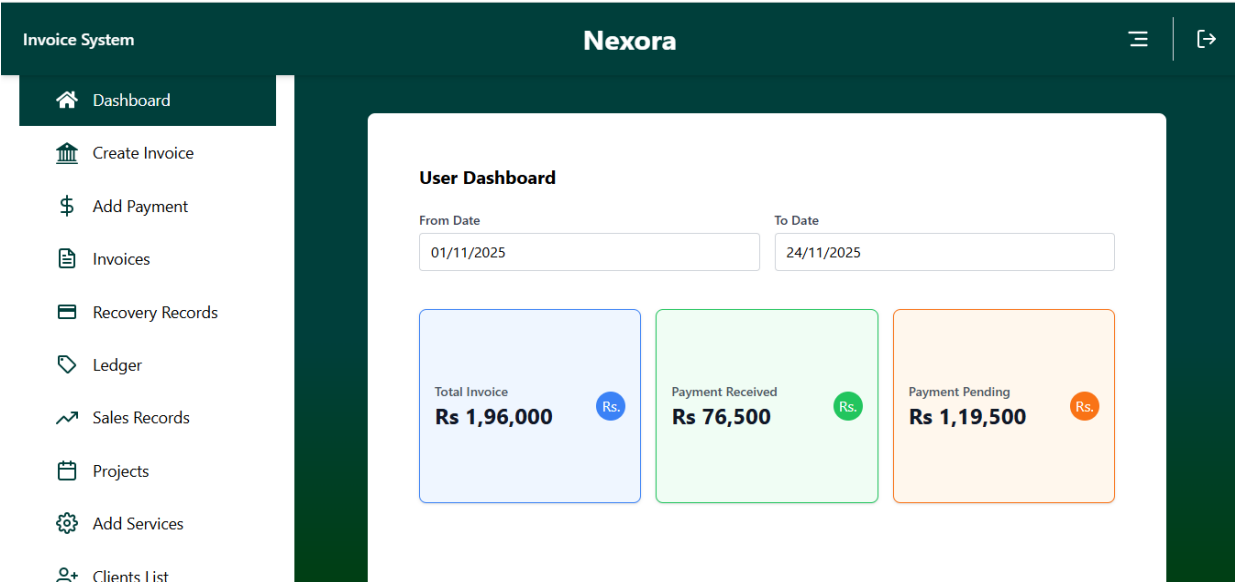


Figure 15 Dashboard

Invoice Creation Screen (Nexora Invoice)

The invoice creation screen allows users to enter customer details, select products, and automatically calculate subtotals and final totals. It provides a clean, form-based layout that makes adding items and generating invoices quick and efficient. Real-time calculations and clear fields ensure accuracy while simplifying the overall billing process.

The screenshot shows the 'Invoice System' form for creating an invoice. It includes a 'Search Customer' field at the top. Below it are input fields for 'Name', 'City', 'Area', and 'Phone'. A 'Date' field is set to '23/11/2025'. There is a 'Search Product' field. Below these is a table with columns: Name, Price, Qty, Subtotal, and Action. At the bottom right, there are summary fields: 'Subtotal: Rs 0.00', 'Flat Discount:' (with an input field), and 'Final Total: Rs 0.00'.

Figure 16 Invoice Creation

## Sales and Record (Nexora Invoice)

The Sales Record screen presents a visual breakdown of product performance using bar charts and pie charts. It allows users to quickly analyze total cartons sold and understand sales distribution across different items. The built-in option to download reports as PDF makes it easy to share and maintain sales documentation for business insights.

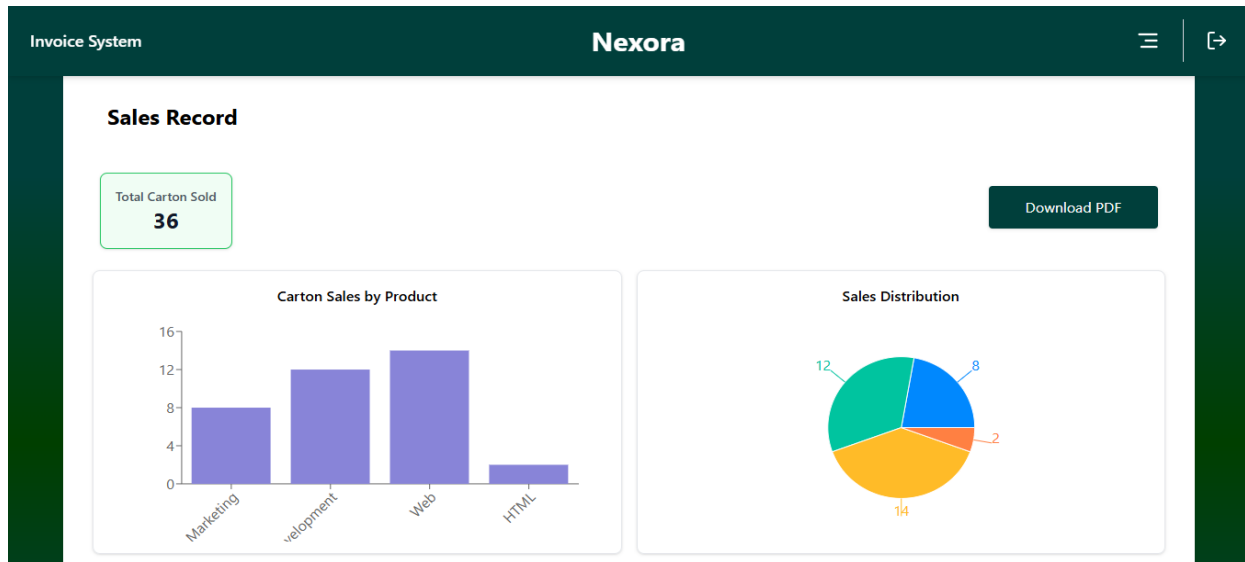


Figure 17 Sales and Record

## Ledger Screen (Nexora Invoice)

The Ledger screen provides a complete financial summary for each customer, including total invoices, payments received, and pending balances. Users can filter records by date, view detailed transaction history, and print full or selected ledger reports. This module helps businesses maintain accurate financial tracking and monitor customer payment behavior efficiently.

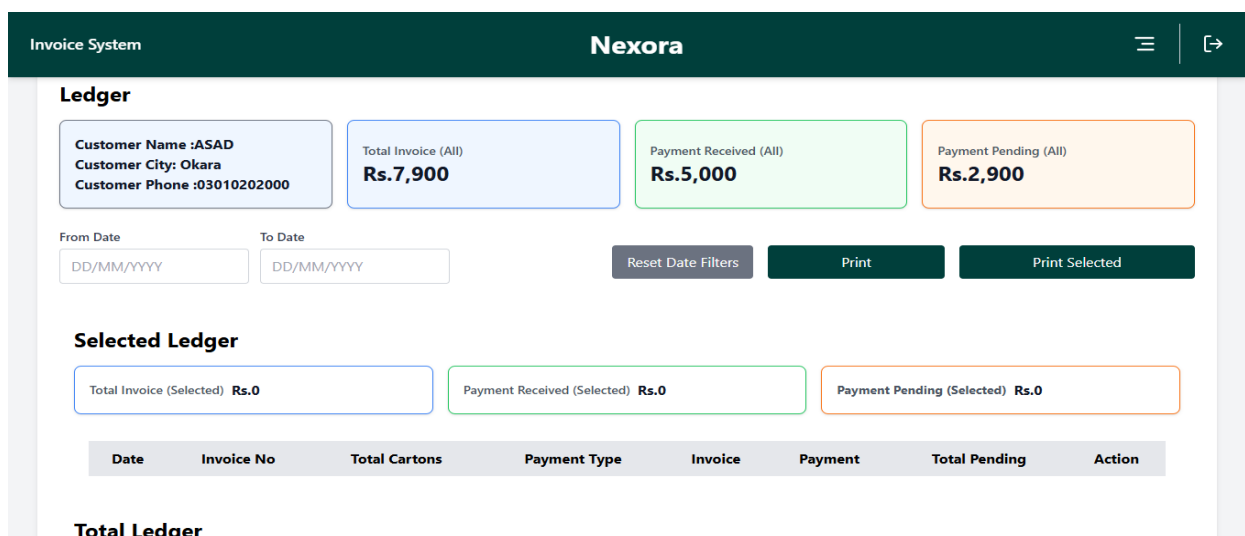
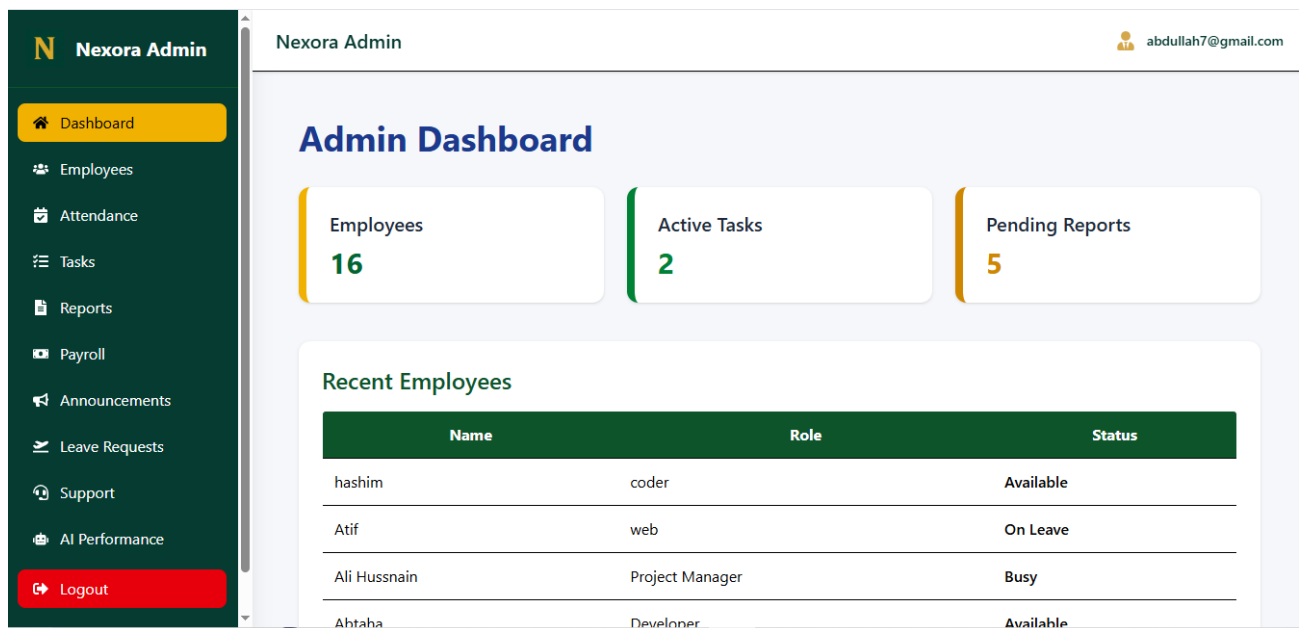


Figure 18 Ledger History

## Nexora Management System

### Admin Dashboard (Nexora EMS)

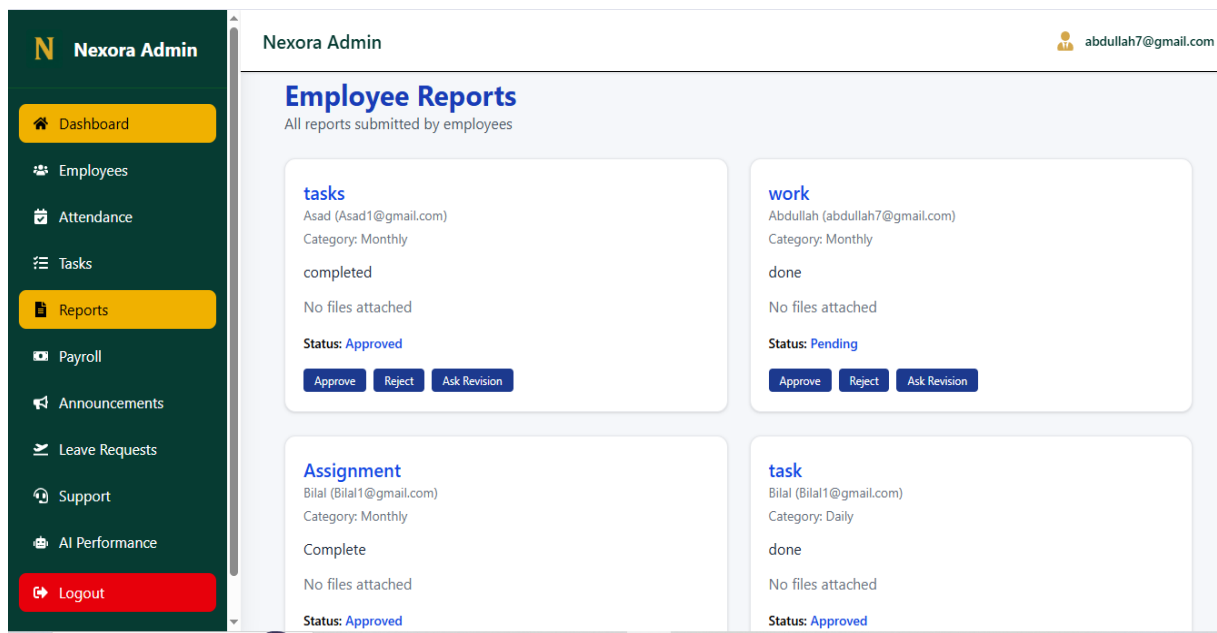
The Admin Dashboard provides a centralized overview of all organizational activities in the Employee Management System. It displays key metrics such as total employees, active tasks, and pending reports, helping administrators monitor overall system performance at a glance. The interface also lists recent employees along with their roles and statuses, enabling quick tracking of workforce availability and workload.



**Figure 19 Admin Panel EMS**

### Employee Reports (Admin Panel)

This screen displays all reports submitted by employees, allowing the admin to monitor work progress, leave requests, and task updates. Each report card shows the employee name, category, details, attachments, and current status. Admins can take actions instantly by approving, rejecting, or requesting revisions. This helps maintain transparency and ensures efficient workflow management within the organization.



**Figure 20 Reports**

## Leave Requests Management (Admin Panel)

This screen allows the administrator to review, approve, or reject leave requests submitted by employees. It clearly displays each employee's leave type, requested dates, reason, and current approval status. The admin can take quick actions using the Approve/Reject buttons, ensuring efficient and transparent leave management.

**Nexora Admin**

**Leave Requests Management**

| Employee                         | Type     | Dates                   | Reason      | Status   | Actions            |
|----------------------------------|----------|-------------------------|-------------|----------|--------------------|
| <b>Asad</b><br>Asad1@gmail.com   | Sick     | 2025-11-28 → 2025-11-29 | fever       | Approved | [Approve] [Reject] |
| <b>Bilal</b><br>Bilal1@gmail.com | Vacation | 2025-11-28 → 2025-11-30 | urgent      | Approved | [Approve] [Reject] |
| <b>Bilal</b><br>Bilal1@gmail.com | Annual   | 2025-11-29 → 2025-11-30 | leave       | Approved | [Approve] [Reject] |
| <b>Bilal</b><br>Bilal1@gmail.com | Annual   | 2025-11-29 → 2025-11-30 | leave       | Approved | [Approve] [Reject] |
| <b>Bilal</b><br>Bilal1@gmail.com | Vacation | 2025-11-28 → 2025-11-28 | enjoy       | Approved | [Approve] [Reject] |
| <b>Bilal</b><br>Bilal1@gmail.com | Unpaid   | 2025-11-27 → 2025-11-28 | need salary | Approved | [Approve] [Reject] |
| <b>Bilal</b><br>Bilal1@gmail.com | Sick     | 2025-11-27 → 2025-11-27 | sick        | Approved | [Approve] [Reject] |

**Figure 21 Leave Requests**

## Intelligent Employee Performance Scoring

The system integrates an AI model that assesses employee activity patterns—such as attendance, tasks, and overall engagement—to produce an automated performance score. These AI insights allow the admin to quickly identify high performers, detect performance drops, and improve management accuracy.

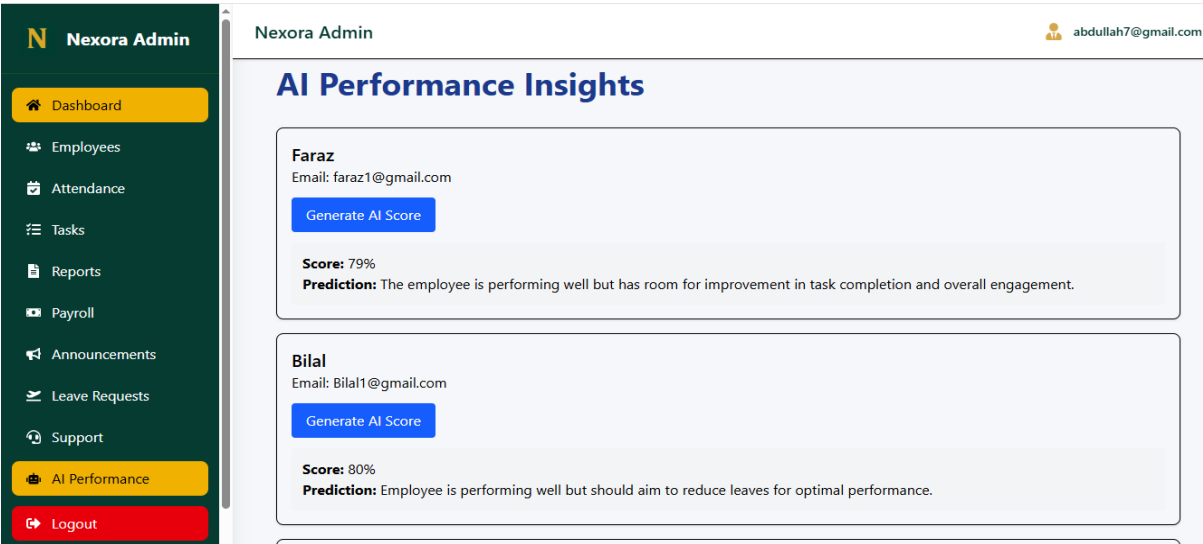


Figure 22 Intelligent Employee Performance Scoring

## User Payslip

The User Payslip screen provides employees with a clear breakdown of their monthly earnings. It displays salary components such as basic pay, bonuses, and deductions in a pie chart, while a line graph shows net salary trends over different months. This helps employees easily understand their financial summary and track salary variations over time.

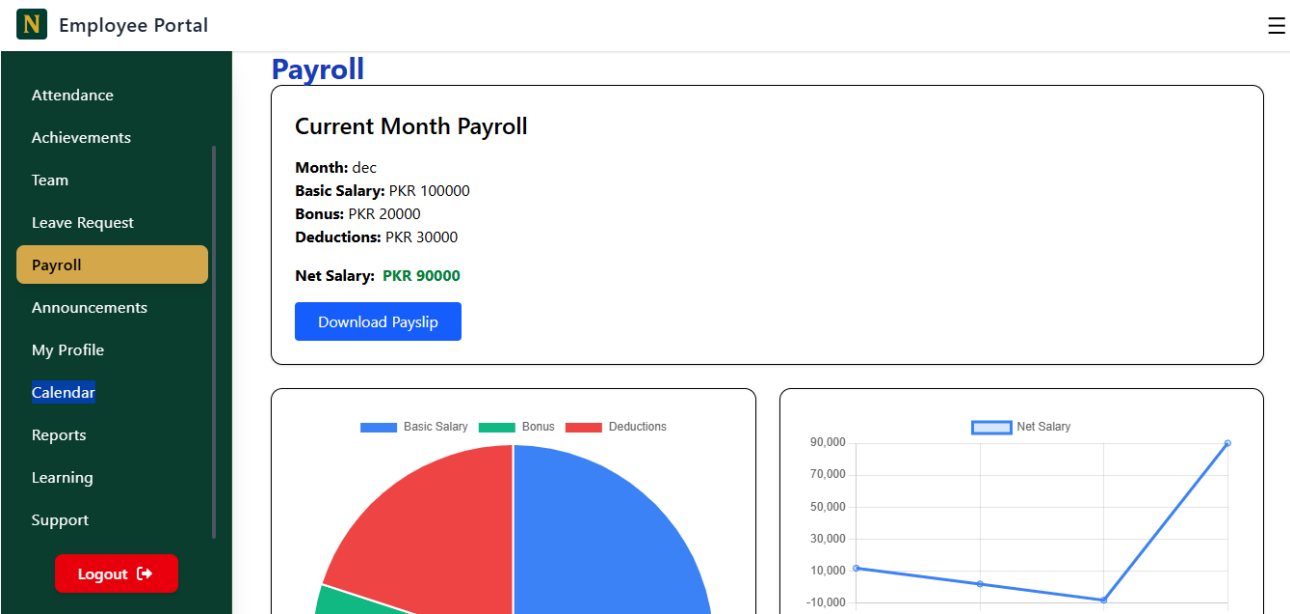


Figure 23 User Payslip



Achievements (User Panel)

The Achievements screen displays a user’s progress and milestones within the organization. Unlocked badges highlight completed accomplishments, while locked achievements show progress toward future goals. This section motivates employees by visually tracking performance and encouraging consistent engagement.

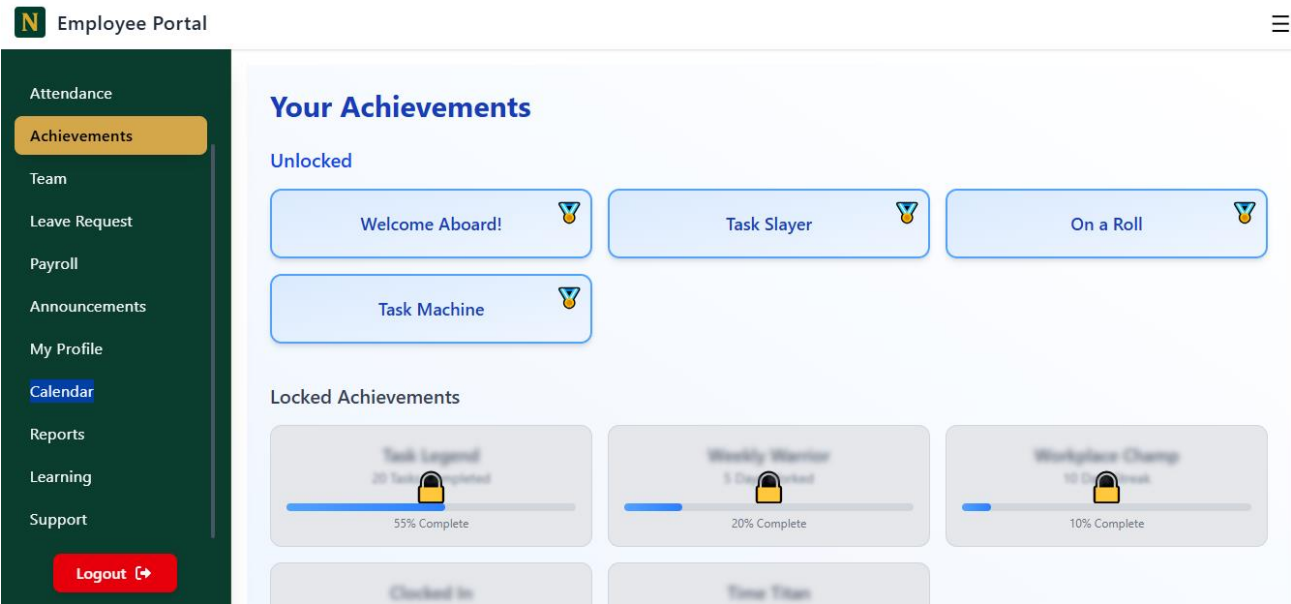


Figure 24 Achievements

Attendance Tracker (User Panel)

The Attendance Tracker allows employees to easily monitor their daily check-in and check-out activity. The screen displays total hours worked for the day along with a monthly calendar showing attendance history. Users can record their presence in real time and view highlighted dates to track their work consistency.

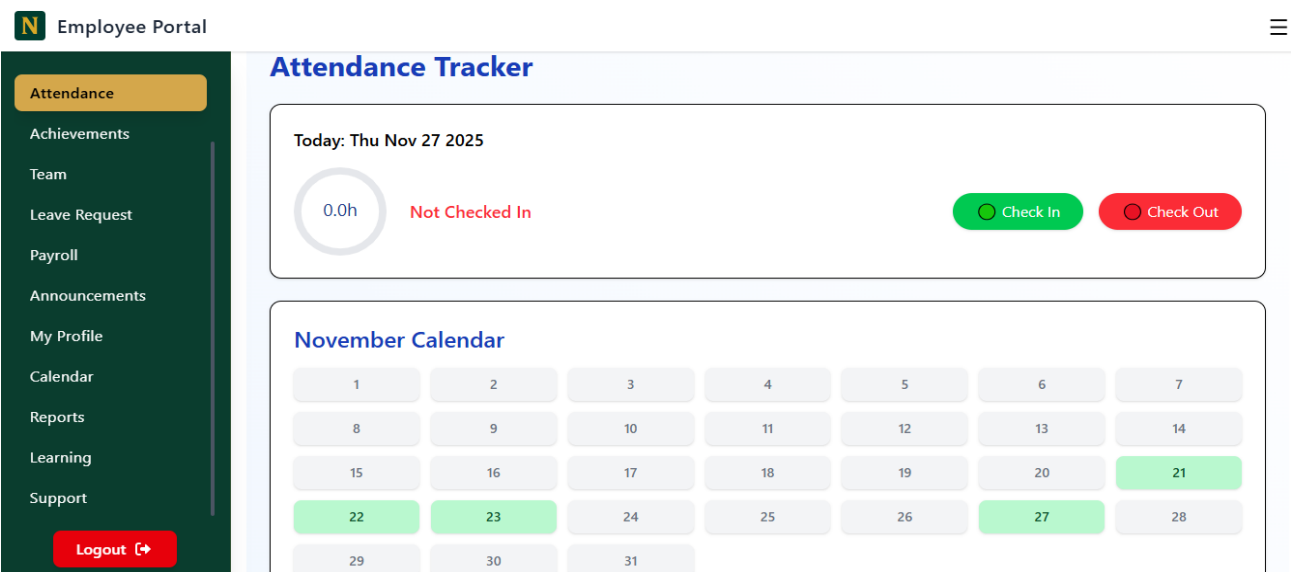


Figure 25 User Attendance

Announcements (Admin Announcements)

The Announcements screen displays all important updates shared by the organization, including general notices, urgent alerts, events, and holiday messages. Each announcement is color-coded by category, allowing employees to quickly identify the type and relevance of the update. This section ensures that users stay informed about schedule changes, important notifications, and upcoming activities in the organization.

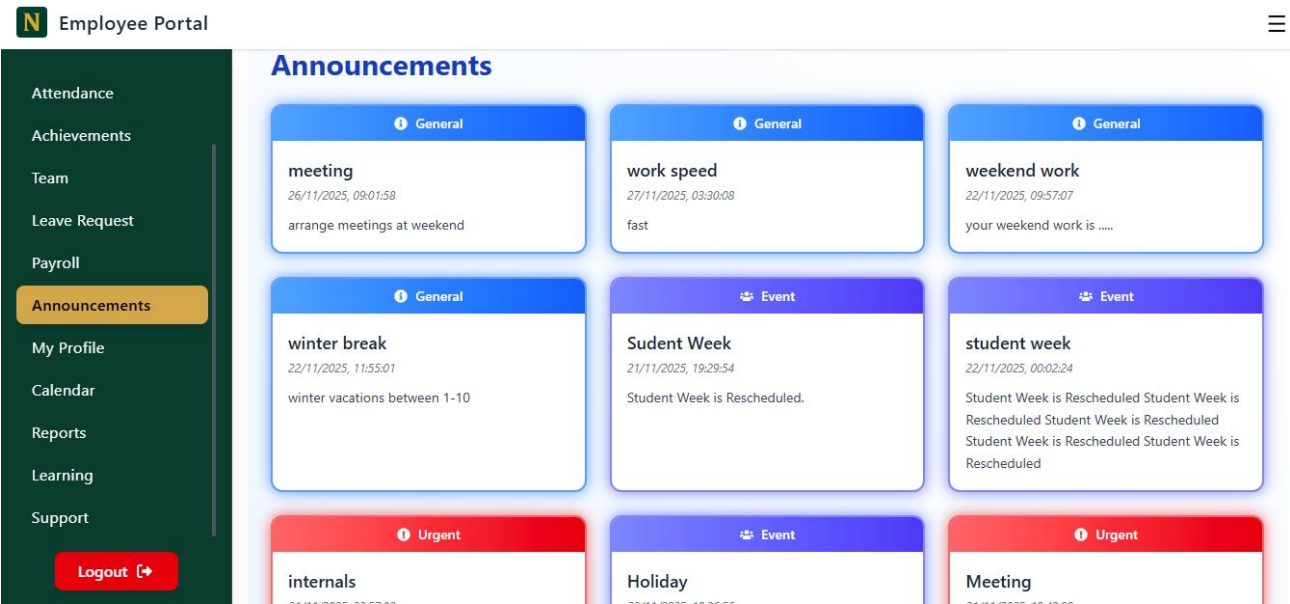


Figure 26 Announcements

# **Chapter 6**

## **Testing and Evaluation**

## 6 Testing and Evaluation

This chapter describes the testing procedures applied to the **Nexora Business Suite**, which includes the Invoice Management System and the Employee Management System. The primary objective of testing was to verify the system's functionality, accuracy, reliability, performance, and interaction between different modules. Each major feature—including user authentication, invoice generation, client and product management, attendance logging, task assignment, payroll processing, and admin operations—was tested thoroughly using standard software testing techniques.

Testing ensured that Nexora meets all functional and non-functional requirements defined in the SRS and SDD. The system was tested under different browsers, devices (desktop and mobile views), and varying internet conditions to observe its behavior in real-world scenarios. Multiple testing strategies were applied, such as **Unit Testing, Functional Testing, Integration Testing, and System Testing**, each validating specific aspects of the web application. These tests helped verify that Nexora performs consistently, handles user workflows smoothly, and maintains data integrity across MongoDB.

### 6.1 Manual Testing

Manual testing involves executing the Nexora system step-by-step to verify that each module operates correctly. This includes testing user authentication, invoice creation, client and product management, attendance logging, task assignment, dashboard updates, and payroll processing. Different testing techniques such as system testing, unit testing, functional testing, and integration testing were applied to ensure that every workflow behaves as expected across MongoDB environments.

#### 6.1.1 System Testing

**Purpose:** To ensure that the entire Nexora Business Suite functions properly as a unified and integrated platform. The goal is to validate that all modules—including Invoice Management, Employee Management, Attendance, Tasks, Payroll, and Dashboard analytics work together smoothly and produce consistent results across MongoDB environments.

**Scope:**

- Verify user login, registration, and role-based access permissions
- Test invoice creation, product/client management, and PDF generation
- Validate employee attendance logging (Check-In / Check-Out)
- Check task assignment, progress updates, and admin monitoring
- Ensure accurate monthly payroll calculations based on attendance data
- Confirm that dashboard metrics update in real-time for both modules
- Validate data consistency between frontend UI and backend databases.

**Test Steps:**

1. Launch the Nexora web application and verify UI loading
1. Register a new user or log in as Admin/Employee

2. Add a client and create a product/service entry
3. Generate an invoice, verify calculations, and download the PDF
4. Mark attendance (Check-In/Out) and verify real-time updates
5. Assign a task to an employee and update its progress
6. Process payroll and confirm salary calculation accuracy
7. Refresh dashboard to ensure updated analytics for all modules

### 6.1.2 Unit Testing

#### Unit Testing 1: Login

**Testing Objective:** To ensure the login system works correctly with role-based access.

*Table 16 Unit Testing 1: Login*

| No | Test case/Test script                       | Attribute and value                           | Expected result / Resu                         | Result |
|----|---------------------------------------------|-----------------------------------------------|------------------------------------------------|--------|
| 1  | Verify login with correct admin credentials | Email: admin@nexora.com<br>Password: admin123 | Login successful → Redirect to Admin Dashboard | Pass   |
| 2  | Verify login with incorrect password        | Email: user@nexora.com<br>Password: wrongPass | System shows: 'Invalid Credentials'            | Pass   |

#### Unit Testing 2: Add Employee

**Testing Objective:** To ensure employee creation works properly in Firestore.

*Table 17 Unit Testing 2: Add Employee*

| No. | Test case/Test script                              | Attribute and value                                       | Expected result               | Result |
|-----|----------------------------------------------------|-----------------------------------------------------------|-------------------------------|--------|
| 1   | Add new employee with valid data                   | Name: Ali Khan<br>Role: Employee<br>Email: ali@Nexora.com | Employee added successfully   | Pass   |
| 2   | Prevent adding employee with empty required fields | Name: " (empty)                                           | System shows validation error | Pass   |

#### Unit Testing 3: Attendance Marking

**Testing Objective:** To ensure attendance Check-In/Check-Out works correctly.

Table 18 Unit Testing 3: Attendance Marking

| No. | Test case/Test script      | Attribute and value     | Expected result                    | Result |
|-----|----------------------------|-------------------------|------------------------------------|--------|
| 1   | Mark Check-In              | User ID: emp01          | Check-In saved in MongoDB          | Pass   |
| 2   | Duplicate Check-In attempt | User already checked-in | System shows: 'Already Checked-In' | Pass   |

#### Unit Testing 4: Create Invoice

**Testing Objective:** To ensure invoice creation, calculation, and saving work correctly.

Table 19 Unit Testing 4: Create Invoice

| No. | Test case/Test script           | Attribute and value             | Expected result                          | Result |
|-----|---------------------------------|---------------------------------|------------------------------------------|--------|
| 1   | Create invoice with valid items | Items: 3<br>Client: XYZ Traders | Invoice generated & stored in MongoDB    | Pass   |
| 2   | Missing product selection       | No items selected               | System shows: 'Select at least one item' | Pass   |

#### Unit Testing 5: Task Assignment.

**Testing Objective:** To ensure admin can assign tasks and save in MongoDB.

Table 20 Unit Testing 5: Task Assignment.

| No. | Test case/Test script   | Attribute and value                     | Expected result                     | Result |
|-----|-------------------------|-----------------------------------------|-------------------------------------|--------|
| 1.  | Assign task to employee | Task: Prepare Report<br>Employee: emp02 | Task assigned successfully          | Pass   |
| 2.  | Missing task title      | Task: " (empty)                         | System shows: 'Task title required' | Pass   |

#### Unit Testing 6: Payroll Calculation

**Testing Objective:** To ensure payroll generation works properly.

Table 21 Unit Testing 6: Payroll Calculation

| No. | Test case/Test script                  | Attribute and value                 | Expected result                          | Result |
|-----|----------------------------------------|-------------------------------------|------------------------------------------|--------|
| 1.  | Generate payroll with valid attendance | Employee: emp05<br>Days Present: 25 | Salary calculated successfully           | Pass   |
| 2.  | Missing attendance records             | Days Present: 0                     | System shows: 'Attendance not available' | Pass   |

## Unit Testing 7: Client Management

**Testing Objective:** To ensure client addition and validation work correctly.

Table 22 Unit Testing 7: Client Management

| No. | Test case/Test script | Attribute and value                            | Expected result                      | Result |
|-----|-----------------------|------------------------------------------------|--------------------------------------|--------|
| 1.  | Add valid client      | Client: ABC Corporation<br>Email: abc@mail.com | Client added successfully            | Pass   |
| 2.  | Invalid email format  | Email: abc@                                    | System shows: 'Invalid Email Format' | Pass   |

## Unit Testing 8: Product/ Service Management

**Testing Objective:** To ensure items/services can be created and validated.

Table 23 Unit Testing 8: Product/ Service Management

| No. | Test case/Test script | Attribute and value                        | Expected result                   | Result |
|-----|-----------------------|--------------------------------------------|-----------------------------------|--------|
| 1.  | Add new service item  | Name: Website Development<br>Price: 50,000 | Item added successfully           | Pass   |
| 2.  | Missing price value   | Price: "                                   | System shows: 'Price is required' | Pass   |

## Unit Testing 9: Task Status Update

**Testing Objective:** Ensure that employees can update task progress correctly and that changes reflect in Fire store and the admin dashboard.

Table 24 Unit Testing 9: Task Status Update

| No. | Test case/Test script               | Attribute and value                   | Expected result                                                         | Result      |
|-----|-------------------------------------|---------------------------------------|-------------------------------------------------------------------------|-------------|
| 1.  | Update task status to "In Progress" | Task ID: T-103<br>Assigned to: emp04  | Status updates to <b>In Progress</b> and reflects in admin view         | <b>Pass</b> |
| 2.  | Update task status to "Completed"   | Task ID: T-103<br>Completed by: emp04 | Status changes to <b>Completed</b> , timestamp saved, dashboard updated | <b>Pass</b> |

### 6.1.3 Functional Testing

#### Functional Testing 1: Login Module

**Objective:** Ensure correct dashboard loads based on Admin/Employee role.

Table 25 Functional Testing 1: Login Module

| No. | Test case/Test script | Attribute and Value                              | Expected Result                            | Result |
|-----|-----------------------|--------------------------------------------------|--------------------------------------------|--------|
| 1   | Login as Admin        | Email: abdullah@nexora.com<br>Password: admin123 | Admin dashboard loaded with full access    | Pass   |
| 2   | Invalid login attempt | Email: emp@nexora.com<br>Password: wrong123      | Error message displayed user not logged in | Pass   |

#### Functional Testing 2: Invoice Generation Module

**Objective:** Ensure invoices are created, calculated, and saved properly.

Table 26 Functional Testing 2: Invoice Generation Module

| No. | Test case/Test script   | Attribute and Value             | Expected Result                                | Result |
|-----|-------------------------|---------------------------------|------------------------------------------------|--------|
| 1   | Create invoice          | Items: 2<br>Client: ABC Traders | Invoice created & stored in MongoDB            | Pass   |
| 2   | Cancel invoice creation | User cancels action             | Invoice not created, redirect to invoices page | Pass   |
| 3   | Invalid input           | No items selected               | Error message displayed                        | Pass   |

#### Functional Testing 3: Attendance Module



**Objective:** Ensure attendance Check-In/Out works correctly.

*Table 27 Functional Testing 3: Attendance Module*

| No. | Test case/Test script | Attribute and Value | Expected Result                | Result |
|-----|-----------------------|---------------------|--------------------------------|--------|
| 1   | Mark Check-In         | User ID: emp01      | Attendance marked successfully | Pass   |
| 2   | Mark Check-Out        | User ID: emp01      | Checkout time updated          | Pass   |
| 3   | Duplicate check-in    | Already checked-in  | Error displayed                | Pass   |

#### Functional Testing 4: Task Assignment Module

**Objective:** To ensure tasks can be assigned and updated correctly.

*Table 28 Functional Testing 4: Task Assignment Module*

| No. | Test case/Test script | Attribute and Value                     | Expected Result            | Result |
|-----|-----------------------|-----------------------------------------|----------------------------|--------|
| 1   | Assign task           | Task: Prepare Report<br>Employee: emp03 | Task assigned successfully | Pass   |
| 2   | Update task status    | Status: Completed                       | Task updated in MongoDB    | Pass   |
| 3   | Unauthorized edit     | Employee editing another user's task    | Access denied              | Pass   |

#### Functional Testing 5: Client Management Module

**Objective:** To ensure clients can be added and validated properly.

*Table 29 Functional Testing 5: Client Management Module*

| No. | Test case/Test script | Attribute and Value                          | Expected Result              | Result |
|-----|-----------------------|----------------------------------------------|------------------------------|--------|
| 1   | Add valid client      | Client: SoftTech<br>Email: info@softtech.com | Client added successfully    | Pass   |
| 2   | Invalid email         | Email: abc@                                  | Validation error displayed   | Pass   |
| 3   | Duplicate client      | Existing client email                        | Error: Client already exists | Pass   |

### Functional Testing 6: Product/Service Module

**Objective:** To Ensure services/products are added and displayed correctly.

Table 30 Functional Testing 6: Product/Service Module

| No. | Test case/Test script | Attribute and Value                  | Expected Result             | Result |
|-----|-----------------------|--------------------------------------|-----------------------------|--------|
| 1   | Add service           | Service: Web Hosting<br>Price: 15000 | Service added successfully  | Pass   |
| 2   | Missing price         | Price: "                             | System shows price required | Pass   |
| 3   | Invalid stock value   | Stock: -5                            | Error displayed             | Pass   |

### Functional Testing 7: Payroll Module

**Objective:** To Ensure payroll calculation and report generation works accurately.

Table 31 Functional Testing 7: Payroll Module

| No. | Test case/Test script | Attribute and Value                 | Expected Result                | Result |
|-----|-----------------------|-------------------------------------|--------------------------------|--------|
| 1   | Generate payroll      | Employee: emp02<br>Days Present: 26 | Salary calculated successfully | Pass   |
| 2   | Missing attendance    | Days present: 0                     | Error displayed                | Pass   |
| 3   | View payroll history  | Employee: emp02                     | Payroll records displayed      | Pass   |

### Functional Testing 8: Dashboard Module

**Objective:** To ensure dashboard graphs, totals, and stats update correctly.

Table 32 Functional Testing 8: Dashboard Module

| No. | Test case/Test script | Attribute and Value | Expected Result             | Result |
|-----|-----------------------|---------------------|-----------------------------|--------|
| 1   | Load dashboard        | Admin user          | All modules stats displayed | Pass   |

|   |                       |                       |                             |      |
|---|-----------------------|-----------------------|-----------------------------|------|
| 2 | Real-time task update | Task marked completed | Dashboard updates instantly | Pass |
| 3 | Invoice update        | New invoice added     | Invoice count updates       | Pass |

### Functional Testing 9: File Upload Module

**Objective:** To ensure uploaded invoice images/documents are stored correctly.

Table 33 Functional Testing 9: File Upload Module

| No. | Test case/Test script | Attribute and Value | Expected Result              | Result |
|-----|-----------------------|---------------------|------------------------------|--------|
| 1   | Upload valid file     | File: invoice.jpg   | File uploaded to Cloudinary  | Pass   |
| 2   | Invalid format        | File: abc.bmp       | Error: Unsupported file type | Pass   |
| 3   | Large file            | File: 12MB          | Upload warning displayed     | Pass   |

#### 6.1.4 Integration Testing

##### Integration Testing 1: Login + Dashboard + Role Access

**Objective:** To ensure correct dashboard loads and modules appear based on Admin/Employee roles.

| No. | Test Case / Script | Modules Integrated              | Expected Result                               | Result |
|-----|--------------------|---------------------------------|-----------------------------------------------|--------|
| 1   | Login as Admin     | Login + Dashboard + Role Access | Admin dashboard loaded with full access       | Pass   |
| 2   | Login as Employee  | Login + Dashboard + Role Access | Employee dashboard loaded with limited access | Pass   |
| 3   | Invalid Login      | Login + Dashboard               | Error shown; user not redirected              | Pass   |

##### Integration Testing 2: Invoice Creation + PDF Generation + Client/Item Module

**Objective:** To ensure invoices are created using client/product modules and exported correctly.

| No. | Test Case / Script                   | Modules Integrated              | Expected Result                      | Result |
|-----|--------------------------------------|---------------------------------|--------------------------------------|--------|
| 1   | Create invoice using existing client | Invoice + Client + MongoDB      | Invoice saved in MongoDB             | Pass   |
| 2   | Generate invoice PDF                 | Invoice Module + PDF Service    | Invoice PDF generated and downloaded | Pass   |
| 3   | Invalid invoice data                 | Invoice Module + Product Module | Error displayed; invoice not created | Pass   |

### Integration Testing 3: Attendance + Payroll Module

**Objective:** To ensure attendance records correctly influence payroll calculation.

| No. | Test Case / Script                    | Modules Integrated               | Expected Result                  | Result |
|-----|---------------------------------------|----------------------------------|----------------------------------|--------|
| 1   | Generate payroll with attendance logs | Attendance + Payroll + Firestore | Salary calculated successfully   | Pass   |
| 2   | No attendance record                  | Attendance + Payroll             | Error: Attendance not found      | Pass   |
| 3   | Realtime payroll update               | Attendance + Payroll + Dashboard | Dashboard updates salary summary | Pass   |

### Integration Test Table 4: Task Assignment + Employee Module + Tracking

**Objective:** To verify tasks assigned to employees update in real-time.

| No. | Test Case / Script       | Modules Integrated            | Expected Result                  | Result |
|-----|--------------------------|-------------------------------|----------------------------------|--------|
| 1   | Assign task              | Task Module + Employee Module | Task saved in MongoDB            | Pass   |
| 2   | Update task to Completed | Task Module + Tracking Module | Task status updated successfully | Pass   |
| 3   | Unauthorized task edit   | Task Module + Role Access     | Access denied                    | Pass   |

### Integration Test Table 5: Client Module + Invoice Report + Dashboard

**Objective:** To ensure client data flows correctly into invoice reporting and dashboard.

| No. | Test Case / Script              | Modules Integrated             | Expected Result          | Result |
|-----|---------------------------------|--------------------------------|--------------------------|--------|
| 1   | Add client and generate invoice | Client Module + Invoice Module | Invoice linked to client | Pass   |

|   |                                    |                                    |                                   |      |
|---|------------------------------------|------------------------------------|-----------------------------------|------|
| 2 | Dashboard invoice summary          | Invoice Module + Dashboard         | Dashboard reflects updated totals | Pass |
| 3 | Delete client with active invoices | Client Module + Invoice Validation | Error shown; client not deleted   | Pass |

### Integration Test Table 6: File Upload + Cloudinary + Invoice Module

**Objective:** To ensure uploaded files store correctly and attach to invoices.

| No. | Test Case / Script      | Modules Integrated          | Expected Result                | Result |
|-----|-------------------------|-----------------------------|--------------------------------|--------|
| 1   | Upload invoice image    | File Upload + Cloudinary    | Image uploaded to Cloudinary   | Pass   |
| 2   | Attach image to invoice | Cloudinary + Invoice Module | Invoice updated with file link | Pass   |
| 3   | Invalid file upload     | File Upload + Validation    | Error displayed; upload failed | Pass   |

## 6.2 Automated Testing

**Tools used:**

Table 34 Automated Testing

| Tool Name                 | Tool Description                                                                                           | Applied On [Test Cases / FR / NFR]                                                 | Results                                                                                 |
|---------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Jest + Supertest          | MERN backend APIs ke liye automated testing framework. REST endpoints aur validation verify karta hai.     | Invoice API, Client CRUD, Authentication, Product Module (FR: UC-01, UC-03, UC-05) | Sab API calls pass. Invoice totals, authentication aur CRUD operations sahi return hue. |
| Firebase Emulator Suite   | Firestore, Auth aur Storage ka local simulation. EMS workflows safe testing environment me run hotay hain. | Attendance Logs, Payroll, Tasks, Role Permissions (FR: UC-07, UC-08, UC-10)        | Attendance, payroll aur role-based access ne har scenario me correct output diya.       |
| Postman Collection Runner | Automated API testing tool jo multiple sequential requests run karta hai.                                  | Invoice Creation, Client/Service Updates, Payment Status, Product Stock            | Responses accurate aur stable. Koi performance issue nahi aya.                          |
| Cloudinary API Tests      | Document aur image file uploads, optimization aur URL delivery test karta hai.                             | Invoice Attachments, Product Images (FR: UC-04, UC-05)                             | Uploads successful, optimized URLs returned. Koi broken upload nahi.                    |
| React Testing Library     | UI components, forms aur navigation test karta hai.                                                        | Login UI, Dashboard, Invoice Form, Product Form                                    | UI rendering, form validations aur navigation sahi perform hue.                         |
| Cypress E2E Tests         | Browser automation tool for full workflow testing.                                                         | Login → Add Client → Create Invoice → Export PDF → Dashboard Update                | Complete user workflow smoothly. execute hua. Data                                      |

|  |  |  |                                       |
|--|--|--|---------------------------------------|
|  |  |  | flow aur module<br>sync correct raha. |
|--|--|--|---------------------------------------|

# **Chapter 7**

## **Conclusion and Future Work**

## 7 Conclusion And Future Work

This chapter presents the overall conclusion of the Nexora Business Suite project, evaluates how successfully it fulfills the defined objectives, and discusses the potential enhancements that can further improve its reliability, scalability, and business value.

### 7.1 Conclusion

The **Nexora Business Suite** successfully delivers a unified business management platform consisting of two integrated systems:

1. **Nexora Invoice (MERN Stack)**
2. **Nexora Employee Management System (MongoDB)**
3. **Pet-Store (MERN Stack)**

Both systems collectively address key business needs such as invoicing, client management, employee tracking, payroll automation, task assignment, file handling, and dashboard analytics.

The system meets the functional and non-functional requirements defined in the SRS and adheres to the architectural and design principles laid out in the SDD. The project achieves its primary objective of simplifying business operations by offering a modern, scalable, and user-friendly solution.

Key achievements include:

#### 1. Robust Invoicing & Client Management

- Automated invoice creation using MERN APIs ensures accuracy and fast processing.
- Real-time product/service selection with auto-calculated totals.
- Cloudinary file uploads allow attaching proof, receipts, and documents to invoices.
- MongoDB ensures secure and structured storage of client and invoice data.

#### 2. Employee Attendance, Payroll & Task Management

- Employees can mark daily attendance using MongoDB.
- Payroll is auto-generated based on attendance logs, ensuring transparency.
- Admins can assign tasks, track progress, and monitor performance in real-time.
- Role-based access ensures proper permissions between Admins and Employees.

#### 3. Real-Time Cloud Integration

- Firestore provides instant syncing for attendance, tasks, and payroll.
- MERN backend ensures high-performance invoice processing.



- Dashboard data (revenue, attendance, tasks, clients) updates dynamically.

#### **4. Enhanced Usability & User Experience**

- A clean React-based UI ensures easy navigation for both Admins and Employees.
- Employee System provides smooth mobile-like behavior like MongoDB and updates.
- Automated workflows reduce manual workload and improve productivity.

#### **5. Secure & Reliable Architecture**

- Authentication handled via JWT (in MERN).
- MongoDB ensure data durability with minimal risk of loss.
- Validation, error-handling, and logging achieved robustly across modules.

### **7.2 Future Work**

Although Nexora Business Suite is fully functional and stable, several improvements can further enhance its capabilities and make it enterprise-level.

#### **1. Mobile Application Development**

- Develop a dedicated Android/iOS app for employees and administrators.
- Provide offline attendance and task access for field employees.

#### **2. Automated Payment Gateway Integration**

- Add JazzCash, Easypaisa, Stripe, or PayPal for direct invoice payments.
- Allow clients to view and pay invoices online.

#### **3. Advanced Analytics & Reporting**

- AI-based analytics for revenue forecasting, employee performance scores, and client trends.
- Exportable reports in PDF/Excel formats.

#### **4. AI-Based Attendance & Task Prediction**

- Face recognition-based attendance.
- AI suggestions for workload balancing among employees.

#### **5. Multi-Branch / Multi-Company Support**

- Allow one admin to manage different branches with separate dashboards.
- Shared employees and resources across companies.

#### **6. Automated Email & WhatsApp Notifications**

- Alerts for new invoices, task assignments, due dates, and payroll summaries.

- Send invoices directly to clients via WhatsApp API.

## **7. Inventory Tracking & Stock Alerts**

- Extend the invoice system to include inventory deduction on each sale.
- Auto-reminders when stock falls below minimum threshold.

## **8. Role-Based Web Portal for Clients**

- Clients can log in, download invoices, view payments, and communicate with the business.

## 8 References

- [1] UML diagrams such as Class Diagrams, Use Case Diagrams, and Sequence Diagrams were used to model the structural and behavioral aspects of the Nexora system. These diagrams helped define relationships among system modules such as invoicing, employee workflows, and resource management. Reference:  
<https://www.geeksforgeeks.org/unified-modeling-language-uml-introduction/>
- [2] Data Flow Diagrams (DFDs) were utilized to visualize key processes such as invoice generation, sales flow, attendance updates, and payroll handling. These diagrams helped explain how data moves across the business suite. Reference:  
<http://www.agilemodeling.com/artifacts/dataFlowDiagram.htm>
- [3] Activity Diagrams were used to represent workflow sequences such as order creation, task assignments, leave requests, and employee login behavior in Nexora. Standard notations referenced from:  
<https://www.geeksforgeeks.org/unified-modeling-language-uml-activity-diagrams/>
- [4] State Transition Diagrams helped describe transitions between various operational states like invoice drafted → approved → paid, or employee check-in → active → check-out. Example reference:  
<http://geeksforgeeks.org/state-transition-diagram-for-an-atm-system/>
- [5] Firebase Authentication was used in the Employee Management System to manage secure login, registration, token verification, and role-based access for admins and employees. Documentation:  
<https://firebase.google.com/docs/auth>
- [6] Cloud Firestore served as a scalable NoSQL database for storing attendance logs, task details, payroll data, employee profiles, and AI-generated performance insights. Reference:  
<https://firebase.google.com/docs/firestore>  
<https://firebase.google.com/docs/cloud-messaging>
- [7] OpenAI APIs were used to generate AI-based employee performance ratings and analytical insights inside Nexora EMS. These APIs enabled text processing and predictive analysis. Reference:  
<https://platform.openai.com/docs>
- [8] Machine Learning concepts and cloud-based AI techniques were referred to design the performance scoring algorithm used in Nexora for employee evaluation and reporting. General ML reference:  
<https://www.tensorflow.org/learn>
- [9] MERN Stack architecture (MongoDB, Express.js, React.js, Node.js) was used to develop Nexora's Invoice Management module for handling orders, invoices, and product data. Documentation:  
<https://www.mongodb.com/docs/>
- [10] React.js documentation was followed to build responsive interfaces for dashboards, invoice screens, sales analytics, and employee panels. UI reference:  
<https://react.dev/>

- [11] Node.js and Express.js were used to build backend APIs for invoice generation, stock updates, sales reports, and customer management. Documentation available at:  
<https://expressjs.com/>
- [12] Material UI and modern UI/UX principles were followed to design user-friendly dashboards, forms, and charts for both Invoice and EMS modules. General reference:  
<https://m2.material.io/design>
- [13] Stripe API documentation was referenced for understanding digital payment processing workflows, which may be integrated into invoice billing and subscriptions. Reference:  
<https://stripe.com/docs>
- [14] Software Performance Testing guidelines were referred to evaluate API load handling, sales filtering performance, and attendance synchronization speed in Nexora. Reference:  
<https://www.geeksforgeeks.org/software-performance-testing/>
- [15] Software Testing Techniques including functional, unit, integration, and system testing were applied to validate Nexora's modules. Documentation:  
<https://www.geeksforgeeks.org/software-testing>
- [16] REST API principles were followed to design secure and scalable API endpoints for products, invoices, employees, and tasks. General reference:  
<https://www.geeksforgeeks.org/rest-api-introduction/>
- [17] UI/UX standards were applied to ensure intuitive navigation across dashboards, employee panels, invoice creation menus, and reporting screens. Reference:  
<https://www.interaction-design.org/literature/topics/ui-design>
- [18] Agile Software Development methodologies were followed to iteratively build, update, and refine both Nexora modules (Invoice System & Employee Management System). Reference:  
<https://www.agilealliance.org/agile101/>